CHULETA FINAL DE JAVASCRIPT

2o DAW - Repaso completo + examen 2_2 | Metodos, sintaxis, patrones y errores tipicos

Basada en los materiales de clase aportados y en el ejercicio 2_2 (tienda.js).

OBJETIVO: Reconocer rapidamente que metodo, estructura o patron necesita cada enunciado, y escribir una solucion segura sin modificar datos cuando el ejercicio lo prohiba.

Como usar esta chuleta

- Primero mira el verbo del enunciado: transformar, filtrar, buscar, comprobar, calcular, ordenar, anadir o eliminar.
- Despues decide si necesitas un valor, un elemento, una posicion, un array nuevo o modificar el array original.
- Por ultimo comprueba mutabilidad, tipos y el valor de retorno del metodo.
- En el ejercicio 2_2, los patrones de Partes 4-6 son prioritarios.

INDICE RAPIDO POR TIPO

Tipo	Metodo	Muta?
Anadir al final	push(...)	Modifica
Quitar del final	pop()	Modifica
Anadir al inicio	unshift(...)	Modifica
Quitar del inicio	shift()	Modifica
Borrar/reemplazar	splice(i,n,...)	Modifica
Copiar parte	slice(i,f)	No
Unir arrays	concat(...)	No
Recorrer sin crear array	forEach(fn)	No
Transformar	map(fn)	No
Filtrar	filter(fn)	No
Calcular/acumular	reduce(fn, inicial)	No
Reducir derecha->izquierda	reduceRight(fn,inicial)	No
Buscar posicion	indexOf(x)	No
Buscar ultima posicion	lastIndexOf(x)	No
Buscar existencia	includes(x)	No
Buscar elemento	find(fn)	No
Buscar posicion por criterio	findIndex(fn)	No
Buscar ultima posicion por criterio	findLastIndex(fn)	No
Comprobar alguno	some(fn)	No
Comprobar todos	every(fn)	No
Ordenar numeros	sort((a,b)=>a-b)	Modifica
Ordenar texto	sort((a,b)=>a.localeCompare(b))	Modifica
Invertir	reverse()	Modifica

Tipo	Metodo	Muta?
Rellenar	fill(v)	Modifica
Aplanar	flat(n)	No
Aplanar+transformar	flatMap(fn)	No
String -> array	split(sep)	No
Array -> string	join(sep)	No
Comprobar array	Array.isArray(v)	No

MAPA DE DECISION: QUE METODO ELEGIR

Enunciado habitual	Eleccion	Esqueleto
"Transforma cada elemento"	map	array.map(x => ...)
"Quedate con los que cumplen"	filter	array.filter(x => condicion)
"Haz algo con cada uno"	forEach	array.forEach(x => ...)
"Encuentra el primero"	find	array.find(x => condicion)
"Dime su indice"	findIndex	array.findIndex(x => condicion)
"Existe este valor"	includes	array.includes(valor)
"Alguno cumple"	some	array.some(x => condicion)
"Todos cumplen"	every	array.every(x => condicion)
"Calcula un total"	reduce	array.reduce((acc,x)=>...,0)
"Ordena"	sort	[...array].sort(comparador) si no se puede mutar
"Primero de la cola"	shift	cola.shift()
"Urgente al principio"	unshift	cola.unshift(x)
"Ultimo de la pila"	pop	pila.pop()
"Anade al carrito"	push	carrito.push(x)
"Elimina por posicion"	indexOf + splice	const i=array.indexOf(x); array.splice(i,1)
"Copia un tramo"	slice	array.slice(inicio,fin)
"Texto separado por..."	split	texto.split(separador)
"Une elementos con..."	join	array.join(separador)
"Arrays anidados"	flat / flatMap	array.flat() / array.flatMap(fn)

1. FUNDAMENTOS QUE TIENES QUE TENER AUTOMATICOS

Variables, tipos y conversiones

Tema	Sintaxis / ejemplo	Recuerda
let	let edad = 18; edad = 19;	Puede reasignarse.
const	const nombre = "Miku";	No puede reasignarse; arrays/objetos pueden mutar.
typeof	typeof 42 // "number"	typeof null es "object" por una particularidad historica.
Array.isArray	Array.isArray(valor)	Es la comprobacion correcta para arrays.

Tema	Sintaxis / ejemplo	Recuerda
undefined	let x; // undefined	Ausencia de valor no inicializado.
null	let x = null;	Ausencia intencionada.
Number	Number("12") // 12	Conversion a numero.
parseInt	parseInt("12") // 12	Entero.
parseFloat	parseFloat("3.14") // 3.14	Decimal.
toString	(12).toString() // "12"	Numero -> string.

Strings y template literals

```
const nombre = "Miku"
nombre.length
const mensaje = `Producto: ${nombre}`
```

REGLA: Para construir mensajes del ejercicio usa preferentemente `...${...}...`; evita concatenaciones largas con +.

Objetos, destructuring y JSON

Concepto	Sintaxis	Uso
Objeto	const p = { nombre: "Zelda", precio: 60 };	Agrupar datos relacionados.
Propiedad	p.nombre / p[clave]	Acceder a datos.
Anadir	p.stock = 3	Agregar propiedad.
Borrar	delete p.stock	Eliminar propiedad.
Copia con cambio	{ ...p, stock: 2 }	Crear objeto nuevo y sobreescribir una propiedad.
Destructuring objeto	const { nombre, precio } = p	Extraer propiedades.
Destructuring array	const [a,b] = valores	Extraer por posicion.
Rest	const [primero, ...resto] = valores	Recoger el resto en un array.
JSON -> JS	JSON.parse(texto)	Convertir texto JSON a dato JS.
JS -> JSON	JSON.stringify(objeto)	Convertir dato JS a texto JSON.

2. ESTRUCTURAS DE CONTROL

Comparaciones y logica

Sintaxis	Significado	Ejemplo
===	Igualdad estricta	"42" === 42 // false
!==	Desigualdad estricta	stock !== 0
< > <= >=	Comparaciones	precio >= 50
&&	Y	stock > 0 && precio > 0
||	O	vip || estudiante
!	Negacion	!activo
??	Valor por defecto si null/undefined	nombre ?? "Sin nombre"
?.	Acceso seguro	usuario?.direccion?.calle

EXAMEN: Usa === y !==. Evita == y !=. Para NaN, usa Number.isNaN(x).

if / else / ternario / switch

```
if (stock > 0) {
  estado = "Disponible"
} else {
  estado = "Agotado"
}

const estado2 = stock > 0 ? "Disponible" : "Agotado"

switch (opcion) {
  case 1: ...; break
  case 2: ...; break
  default: ...
}
```

Bucles

Estructura	Cuando	Esqueleto
while	Repetir mientras se cumpla una condicion	while (condicion) { ... }
do...while	Necesitas al menos una vuelta	do { ... } while (condicion)
for	Controlar un indice/contador	for (let i=0; i<arr.length; i++) { ... }
for...of	Recorrer valores	for (const x of array) { ... }
for...in	Recorrer claves	for (const clave in objeto) { ... }
break	Salir del bucle	if (cond) break
continue	Saltar a siguiente vuelta	if (cond) continue

REGLA: for...of -> valores. for...in -> claves. En Part 6.1, piensa while + shift para procesar una cola.

3. FUNCIONES Y PROGRAMACION FUNCIONAL

Funciones

```
function sumar(a, b) {
  return a + b
}

const sumar2 = (a, b) => a + b
const doble = x => x * 2
const saludo = () => "Hola"
```

Concepto	Que significa	Ejemplo
Parametro	Nombre que recibe la funcion	function f(x) {} -> x
Argumento	Valor que pasas al llamar	f(10) -> 10
return	Devuelve y termina la funcion	return resultado
Flecha sin llaves	Return implicito	x => x * 2

Concepto	Que significa	Ejemplo
Flecha con llaves	Necesita return	x => { return x * 2 }
Valor por defecto	Se usa si llega undefined	(x = 0) => x + 1
Rest	Recoge argumentos	(primero, ...resto) => ...
Spread	Expande un array	Math.max(...numeros)

IDEA CLAVE: map, filter, find, some, every y reduce reciben funciones. Las funciones son valores de primera clase.

4. METODOS DE ARRAYS - FICHAS DE EXAMEN

push(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.push(...items)	Anadir al final.	Nueva longitud	SI

```
carrito.push("Zelda")
```

USO CONJUNTO: push -> cola; push + shift = FIFO.

ERROR TIPICO: Confundir push con unshift.

pop() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.pop()	Quitar el ultimo.	Elemento eliminado / undefined	SI

```
const ultimo = pila.pop()
```

USO CONJUNTO: push + pop = pila LIFO.

ERROR TIPICO: No usar para quitar el primero.

unshift(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.unshift(...items)	Anadir al principio.	Nueva longitud	SI

```
cola.unshift(pedidoUrgente)
```

USO CONJUNTO: unshift + pop tambien puede implementar una cola.

ERROR TIPICO: Usarlo cuando el enunciado diga urgente/primero.

shift() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.shift()	Quitar el primero.	Elemento eliminado / undefined	SI

```
const siguiente = cola.shift()
```

USO CONJUNTO: En 5.1 y 6.1: cola FIFO.

ERROR TIPICO: No confundir con pop().

splice(i,n,...) - EDITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.splice(indice, cuantos, ...nuevos)	Borrar, insertar o reemplazar en el medio.	Array con lo eliminado	SI

```
carrito.splice(posicion, 1)
arr.splice(2, 0, "nuevo")
```

USO CONJUNTO: indexOf + splice = eliminar la primera aparicion.

ERROR TIPICO: splice modifica el original; slice no.

slice(i,f) - COPIAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.slice(inicio, fin)	Copiar un tramo o hacer una copia completa.	Array nuevo	NO

```
const copia = array.slice()
const trozo = array.slice(1, 3)
```

USO CONJUNTO: slice + sort permite ordenar una copia.

ERROR TIPICO: El fin NO se incluye.

concat(...) - UNIR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.concat(...items)	Unir arrays sin mutar.	Array nuevo	NO

```
const todos = a.concat(b)
```

USO CONJUNTO: Alternativa: [...a, ...b].

ERROR TIPICO: concat no modifica los originales.

5. METODOS DE BUSQUEDA

indexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.indexOf(valor)	Saber la primera posicion de un valor concreto.	Indice o -1	NO

```
const i = carrito.indexOf("Zelda")
```

USO CONJUNTO: indexOf + splice para borrar una aparicion.

ERROR TIPICO: Busca por valor concreto; usa find para objetos/criterios.

lastIndexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.lastIndexOf(valor)	Saber la ultima posicion de un valor concreto.	Indice o -1	NO

```
historial.lastIndexOf("añadir")
```

USO CONJUNTO: Muy util en deshacer cuando hay repetidos.

ERROR TIPICO: No devuelve el elemento.

includes(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.includes(valor)	Saber solo si un valor existe.	true / false	NO

```
["Ana","Luis"].includes("Luis")
```

USO CONJUNTO: Tambien puede comprobar existencia tras map.

ERROR TIPICO: No sirve para buscar con una condicion.

find(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.find(x => condicion)	Encontrar el primer elemento que cumple.	Elemento o undefined	NO

```
catalogo.find(p => p.nombre === "Zelda")
```

USO CONJUNTO: find + propiedad; find para luego leer/modificar un objeto encontrado.

ERROR TIPICO: Puede devolver undefined: no accedas a propiedades sin comprobar.

findIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findIndex(x => condicion)	Encontrar la posicion del primer elemento que cumple.	Indice o -1	NO

```
const i = catalogo.findIndex(p => p.nombre === "Zelda")
```

USO CONJUNTO: findIndex + splice para eliminar por criterio.

ERROR TIPICO: Si no existe, devuelve -1.

findLastIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findLastIndex(x => condicion)	Buscar la ultima posicion que cumple un criterio.	Indice o -1	NO

```
historial.findLastIndex(x => x.tipo === "quitar")
```

USO CONJUNTO: Alternativa especializada cuando se quiere la ultima coincidencia.

ERROR TIPICO: Es una extension de 02.2, no el patron central de tienda.js.

6. TRANSFORMAR, FILTRAR Y CALCULAR

map(fn) - TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.map(x => resultado)	Crear un array nuevo transformando cada elemento.	Array nuevo del mismo tamaño	NO

```
catalogo.map(p => p.nombre)
```

USO CONJUNTO: filter(...).map(...), map(...).join("\n"), map(...).flat()

ERROR TIPICO: No es para filtrar: siempre genera una salida por elemento.

filter(fn) - FILTRAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.filter(x => condicion)	Quedarte con todos los elementos que cumplen.	Array nuevo	NO

```
catalogo.filter(p => p.stock > 0)
```

USO CONJUNTO: filter + map es el patron mas tipico.

ERROR TIPICO: No devuelve un unico objeto.

forEach(fn) - RECORRER

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.forEach((x,i,arr) => ...)	Ejecutar una accion por elemento sin crear resultados.	undefined	NO*

```
productos.forEach(p => console.log(p.nombre))
```

USO CONJUNTO: Usalo cuando el resultado es un efecto (console.log, push, etc.).

ERROR TIPICO: Si necesitas un array, usa map; forEach devuelve undefined.

reduce(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduce((acc, x) => ..., inicial)	Acumular hasta obtener un unico valor.	Un valor	NO

```
const total = productos.reduce((s,p) => s + p.precio, 0)
```

USO CONJUNTO: reduce + find; reduce para totales; reduce con objeto acumulador.

ERROR TIPICO: Especifica valor inicial; sin el, [] produce error.

reduceRight(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduceRight((acc,x) => ..., inicial)	Reducir empezando por la derecha.	Un valor	NO

```
["a","b","c"].reduceRight((t,x) => t+x, "")
```

USO CONJUNTO: 02.2 lo presenta como variante de reduce.

ERROR TIPICO: No confundir con reverse + reduce.

some(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.some(x => condicion)	Comprobar si al menos uno cumple.	true / false	NO

```
catalogo.some(p => p.stock === 0)
```

USO CONJUNTO: some = "hay alguno"; ideal para hayAgotados.

ERROR TIPICO: No significa todos.

every(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.every(x => condicion)	Comprobar si todos cumplen.	true / false	NO

```
pedido.lineas.every(linea => linea.cantidad > 0)
```

USO CONJUNTO: every = "todos"; clave en puedeServirse y preciosValidos.

ERROR TIPICO: Asegurate de recorrer el array correcto.

7. ORDENAR Y METODOS EXTRA

sort(fn) - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.sort((a,b) => ...)	Ordenar elementos.	Array ordenado	SI

```
[...productos].sort((a,b) => a.precio - b.precio)
```

USO CONJUNTO: Si el enunciado no permite mutar: [...array].sort(...).

ERROR TIPICO: Sin comparador, los numeros se comparan como texto.

localeCompare - ORDENAR TEXTO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
a.localeCompare(b)	Ordenar cadenas respetando el idioma/acento.	Numero <0, 0, >0	NO

```
productos.sort((a,b) => a.nombre.localeCompare(b.nombre))
```

USO CONJUNTO: sort + localeCompare para nombres.

ERROR TIPICO: No uses a > b si el ejercicio habla de orden alfabetico con acentos.

reverse() - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reverse()	Invertir el orden.	Array invertido	SI

```
[...array].reverse()
```

USO CONJUNTO: sort ascendente + reverse para descendente, si el ejercicio lo plantea.

ERROR TIPICO: Tambien muta: copia primero si hace falta.

fill(v,i,f) - RELLENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.fill(valor, inicio, fin)	Rellenar posiciones.	Array	SI

```
new Array(5).fill("■")
```

USO CONJUNTO: En graficoStock: new Array(n).fill("■").join("").

ERROR TIPICO: Modifica el array.

flat(n) - APLANAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flat(profundidad)	Aplanar arrays anidados.	Array nuevo	NO

```
[["A","B"],["C"]].flat()
```

USO CONJUNTO: map + flat aparece en cadenas de procesamiento.

ERROR TIPICO: flat() por defecto aplana un nivel.

flatMap(fn) - APLANAR + TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flatMap(x => resultadoArray)	Transformar y aplanar un nivel en una sola operacion.	Array nuevo	NO

```
["hola mundo"].flatMap(x => x.split(" "))
```

USO CONJUNTO: Alternativa compacta a map(...).flat().

ERROR TIPICO: Aplana un solo nivel.

split(sep) - STRING -> ARRAY

Sintaxis	Cuando usarlo	Que devuelve	Muta?
texto.split(separador)	Separar una cadena por un delimitador.	Array	NO

```
"Ana|Pan:2;Leche:1".split("|")
```

USO CONJUNTO: 4.1: split("|") -> split(";") -> split(":").

ERROR TIPICO: split siempre trabaja sobre strings.

join(sep) - ARRAY -> STRING

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.join(separador)	Unir elementos en un texto.	String	NO

```
["Pan","Leche"].join("\n")
```

USO CONJUNTO: 4.5: map + join("\n") para generar ticket.

ERROR TIPICO: No confundir con split.

Array.isArray(v) - TIPO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
Array.isArray(valor)	Comprobar si un valor es un array.	true / false	NO

```
Array.isArray(matriz)
```

USO CONJUNTO: 1.1 y validaciones.

ERROR TIPICO: typeof [] devuelve "object".

toSorted(fn) - ORDENAR SIN MUTAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.toSorted(comparador)	Ordenar sin tocar el original (ES2023).	Array nuevo	NO

```
const ordenados = numeros.toSorted((a,b) => a-b)
```

USO CONJUNTO: Alternativa moderna a [...array].sort(...).

ERROR TIPICO: Tu ejercicio 2_2 trabaja con ES6 y puede exigir spread/slice.

8. TABLA VITAL: MUTA O NO MUTA

Categoria	Metodos / patron
SI mutan	push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin
NO mutan	slice, concat, map, filter, reduce, reduceRight, find, findIndex, findLastIndex, indexOf, lastIndexOf, includes, some, every, join, flat, flatMap
Copia tipica	[...array] o array.slice()
Objeto copia	{ ...objeto, propiedad: nuevoValor }

EXAMEN 2_2: Excepto Parte 5, las funciones no deben modificar los arrays de entrada. Para ordenar, copia antes con [...array] o slice().

9. CUANDO USAR METODOS JUNTOS: RECETAS

Combinacion	Idea	Patron	Uso en examen
filter + map	Seleccionar y despues transformar	productos.filter(p => p.stock > 0).map(p => p.nombre)	Nombres de productos con stock.
map + join	Transformar y construir texto	lineas.map(l => `${l.nombre}: ${l.cantidad}`).join("\n")	Ticket / listado.
split + map	Parsear una cadena a datos	texto.split(";").map(x => x.split(":"))	Pedidos.
find + propiedad	Buscar un objeto y usar un dato	const p = catalogo.find(x => x.nombre === nombre); p.precio	Buscar precio/stock.
findIndex + splice	Eliminar el primer elemento que coincide	const i = a.findIndex(x => x.id === id); if (i !== -1) a.splice(i,1)	Borrar por criterio.
indexOf + splice	Eliminar un valor concreto	const i = a.indexOf(valor); if (i !== -1) a.splice(i,1)	Carrito Parte 5.
reduce + find	Acumular usando datos de objetos	lineas.reduce((t,l) => t + buscarProducto(catalogo,l.nombre).precio*l.cantidad, 0)	Total de pedido.
map + flat + filter + indexOf + sort	Transformar, aplanar, quitar repetidos y ordenar	pedidos.map(p => p.lineas).flat().map(l => l.nombre).filter((n,i,a) => a.indexOf(n) === i).sort((a,b)=>a.localeCompare(b))	productosVendidos 6.2.
[...] + sort	Ordenar sin mutar	[...catalogo].sort((a,b) => a.precio-b.precio)	ordenarPorPrecio / 3 mas baratos.
while + shift	Procesar una cola hasta vaciarla	while (cola.length > 0) { const p = cola.shift(); ... }	procesarCola 6.1.
new Array + fill + join	Crear una barra visual	new Array(Math.floor(stock/2)).fill("■").join("")	graficoStock 6.3.

10. EJERCICIO 2_2 - GUIA DE PATRONES POR PARTES

PRIORIDAD MAXIMA: Partes 4, 5 y 6. Aprende el patron, no memorices la solucion completa.

Parte 4 - Pedidos

Funcion	Que pide	Patron mental	Metodos clave
4.1 parsearPedido	Texto -> objeto pedido	capas de split + map + Number	split, map, Number
4.2 puedeServirse	Todos existen y tienen stock suficiente	pedido.lineas.every -> buscarProducto -> stock >= cantidad	every, find
4.3 totalPedido	Total euros	reduce -> buscarProducto -> precio*cantidad	reduce, find
4.4 servirPedido	Nuevo catalogo con stock descontado	catalogo.map -> find linea -> { ...producto, stock: ... }	map, find, spread
4.5 generarTicket	Texto final con saltos de linea	map -> template literal -> join("\n")	map, join, template literal

```
// 4.1 esquema
const partes = texto.split("|")
const cliente = partes[0]
const lineas = partes[1].split(";").map(linea => {
  const p = linea.split(":")
  return { nombre: p[0], cantidad: Number(p[1]) }
})
```

```
// 4.2 esquema
return pedido.lineas.every(linea => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return producto !== undefined && producto.stock >= linea.cantidad
})

// 4.3 esquema
return pedido.lineas.reduce((total, linea) => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return total + producto.precio * linea.cantidad
}, 0)
```

Parte 5 - Arrays mutables

Funcion	Idea	Metodo	Patron
5.1 atenderSiguiente	Sale el primero	shift	const pedido = cola.shift()
5.2 agregarUrgente	Entra por delante	unshift	cola.unshift(pedido)
5.3 agregarAlCarrito	Entra al final + historial	push	carrito.push(producto)
5.4 quitarDelCarrito	Buscar primera posicion y borrar	indexOf + splice	const i = carrito.indexOf(x); if (i !== -1) carrito.splice(i,1)
5.5 deshacer	Deshacer ultima accion	pop + reverse action + lastIndexOf/splice	const accion = historial.pop(); ...

PISTA: Parte 5 trata al historial como una pila LIFO: la ultima accion registrada es la primera que se deshace.

Parte 6 - Integracion

Funcion	Secuencia	Clave
6.1 procesarCola	while -> shift -> puedeServirse -> servirPedido -> reasignar catalogo	Cola + condicion + inmutabilidad
6.2 productosVendidos	map -> flat -> map -> filter + indexOf -> sort + localeCompare	Pipelines encadenados
6.3 graficoStock	calcular n -> new Array(n).fill("■").join("")	String visual desde un array

```
// 6.1 patron
while (cola.length > 0) {
  const pedido = atenderSiguiente(cola)
  if (puedeServirse(catalogo, pedido)) {
    catalogo = servirPedido(catalogo, pedido)
  }
}
```

11. CHECKLIST DE EXAMEN: ANTES DE ENTREGAR

- [ ] He leido exactamente que pide el enunciado y que array debo recorrer.
- [ ] He elegido map/filter/find/findIndex/some/every/reduce segun el verbo.
- [ ] He usado === y !==, no ==/!=.
- [ ] He comprobado undefined cuando find puede no encontrar nada.
- [ ] He comprobado -1 cuando uso indexOf/findIndex antes de splice.
- [ ] He usado Number() cuando un numero viene de split().
- [ ] He usado localeCompare para texto cuando el orden alfabetico importa.
- [ ] He puesto comparador numerico en sort((a,b)=>a-b).
- [ ] He copiado antes de sort/reverse si no puedo mutar la entrada.

- [ ] He recordado que const no congela arrays/objetos.
- [ ] He usado shift para sacar el primero y push/unshift segun donde entra.
- [ ] He distinguido slice (copia) de splice (modifica).
- [ ] He distinguido split (string -> array) de join (array -> string).
- [ ] En flechas con llaves he puesto return.
- [ ] En reduce he puesto un valor inicial, normalmente 0 para sumas.
- [ ] En 4.2 recorro pedido.lineas, no catalogo.
- [ ] En 4.4 creo objetos nuevos con {...producto, stock: ...}.
- [ ] En 6.1 uso while(cola.length > 0).
- [ ] En 6.2 reconozco la cadena map/flat/map/filter/indexOf/sort.
- [ ] No he cambiado nombres ni parametros que el profesor haya fijado.

12. LA SUPER-CHULETA DE 30 SEGUNDOS

Pregunta del enunciado	Respuesta
Transformar cada elemento	map
Seleccionar varios	filter
Hacer algo por cada uno	forEach
Buscar un elemento	find
Buscar una posicion	findIndex
Buscar si existe	includes
Existe alguno	some
Cumplen todos	every
Calcular un total	reduce
Ordenar numeros	sort((a,b)=>a-b)
Ordenar textos	sort((a,b)=>a.localeCompare(b))
Anadir final	push
Quitar final	pop
Anadir inicio	unshift
Quitar inicio	shift
Eliminar por posicion	splice
Copiar	slice / [...array]
Texto -> array	split
Array -> texto	join
Array -> array plano	flat
Copia objeto + cambio	{ ...objeto, prop: valor }
Todos los pedidos hasta vaciar cola	while + shift
Ultima accion primero	pop (LIFO)
Primero que llega primero sale	shift (FIFO)

MIKU: No intentes recordar 50 soluciones distintas. Reconoce 15-20 patrones y combina los metodos que ya conoces.

Documento de estudio elaborado a partir de los materiales aportados para el repaso del examen.

AMPLIACION: ejemplos de clase + como construir las soluciones

Estas paginas se anaden a la chuleta original: las 15 paginas anteriores se conservan sin cambios. Aqui se incorpora lo que faltaba para estudiar mejor: ejemplos de ejercicios, resultado esperado, patrones de combinacion y una lectura guiada de las funciones.

INDICE DE LA AMPLIACION

A. Como leer y escribir una funcion
B. Como crear y recorrer arrays y objetos
C. Tabla maestra: que pregunta responde cada metodo
D. Agregar, quitar, slice y splice con ejemplos
E. Busqueda y validacion
F. Transformacion: map, filter, forEach
G. Calculo: reduce y reduceRight
H. Ordenar, split, join y flat
I. Metodos extra del nuevo tema 2.2
J. Mutabilidad, spread, copias y metodos encadenados
K. Ejemplos exactos del ejercicio 2.2 - Parte 4
L. Ejemplos exactos del ejercicio 2.2 - Parte 5
M. Ejemplos exactos del ejercicio 2.2 - Parte 6
N. Patrones de examen y errores que cuestan puntos

A. COMO CREAR UNA FUNCION Y COMO LEERLA

Idea: una funcion recibe datos, hace un trabajo y puede devolver un resultado. En tu examen casi todas aparecen como funciones flecha y con return cuando hay que devolver algo.

```
const sumar = (a, b) => { return a + b }
```

Como leerla: sumar es el nombre; a, b son parametros; en la llamada sumar(2, 3), 2 y 3 son argumentos; return devuelve 5.

```
const resultado = sumar(2, 3) // resultado -> 5
```

Flecha corta: cuando solo devuelves una expresion, puedes quitar llaves y return.

```
const doble = n => n * 2 // doble(5) -> 10
```

Con bloque: si pones llaves, necesitas return para devolver un valor.

```
const doble = n => { const resultado = n * 2 return resultado }
```

Funcion que procesa un array: aqui aparece la idea que luego usan map, filter, find, every, etc.: pasas una funcion como argumento.

```
const mayores = numeros => numeros.filter(n => n > 10)
```

Resultado: mayores([4, 12, 20, 3]) -> [12, 20]

B. COMO CREAR ARRAYS, OBJETOS Y ARRAYS DE OBJETOS

Array: una lista ordenada. Los indices empiezan en 0. length indica la cantidad de elementos.

```
const juegos = ['Zelda', 'Persona', 'Mario'] juegos[0] // 'Zelda' juegos.length // 3 juegos[juegos.length - 1] // 'Mario'
```

Objeto: un conjunto de propiedades nombre/valor.

```
const producto = { nombre: 'Zelda', precio: 60, stock: 4 } producto.nombre producto['precio']
```

Array de objetos: es la estructura central de tienda.js. Piensa siempre: primero el array, despues cada objeto y despues la propiedad.

```
const catalogo = [ { nombre: 'Pan', precio: 2, stock: 5 }, { nombre: 'Leche', precio: 1.5, stock: 0 } ]
catalogo[0].nombre // 'Pan'
```

Patron al usar metodos: en map, filter, find, etc., el parametro suele llamarse producto o linea y representa el elemento actual.

```
catalogo.map(producto => producto.nombre) // ['Pan', 'Leche']
```

C. TABLA MAESTRA: QUE PREGUNTA RESPONDE CADA METODO

Pregunta del enunciado	Metodo	Sintaxis base	Resultado
Transforma cada elemento	map	arr.map(x => ...)	array nuevo
Quedate con los que...	filter	arr.filter(x => condicion)	array nuevo
Haz algo por cada uno	forEach	arr.forEach(x => ...)	undefined
Busca un elemento	find	arr.find(x => condicion)	elemento/undefined
Busca su posicion	findIndex	arr.findIndex(x => condicion)	indice/-1
Busca desde el final	findLastIndex	arr.findLastIndex(x => condicion)	indice/-1
Existe ese valor	includes	arr.includes(valor)	true/false
Primera posicion	indexOf	arr.indexOf(valor)	indice/-1
Ultima posicion	lastIndexOf	arr.lastIndexOf(valor)	indice/-1
Hay alguno	some	arr.some(x => condicion)	true/false
Todos	every	arr.every(x => condicion)	true/false
Calcula un unico resultado	reduce	arr.reduce((acc,x)=>..., inicial)	valor
Reduce de derecha a izquierda	reduceRight	arr.reduceRight((acc,x)=>..., inicial)	valor
Ordena	sort	arr.sort((a,b)=>a-b)	array ordenado (muta)
Invierte	reverse	arr.reverse()	array invertido (muta)
Copia un tramo	slice	arr.slice(inicio, fin)	array nuevo
Borra/inserta en una posicion	splice	arr.splice(i,n,...)	array modificado
Une arrays	concat	a.concat(b)	array nuevo
Array -> texto	join	arr.join(separador)	string
Texto -> array	split	texto.split(separador)	array
Aplana arrays anidados	flat	arr.flat(profundidad)	array nuevo
Rellena posiciones	fill	arr.fill(valor,inicio,fin)	array modificado

Atajo mental: map = transformar; filter = seleccionar; find = uno; findIndex = posicion; some = alguno; every = todos; reduce = total; sort = ordenar. Esta traduccion es exactamente la que mas te interesa en el examen.

D. AGREGAR, QUITAR, SLICE Y SPLICE

Los cuatro extremos: push/unshift agregan; pop/shift quitan. push/pop trabajan al final; unshift/shift al principio.

```
const cola = ['Ana', 'Luis'] cola.push('Eva') // ['Ana','Luis','Eva'] cola.unshift('Marta') //
['Marta','Ana','Luis','Eva'] const ultimo = cola.pop() // 'Eva' const primero = cola.shift() // 'Marta'
```

splice: "cirugia". Sirve para borrar, insertar o reemplazar. Su firma es splice(indice, cuantos, ...nuevos).

```
const a = ['Yo', 'estudio', 'JS'] a.splice(1, 1) // ['Yo', 'JS'] a.splice(1, 0, 'aprendo') // ['Yo', 'aprendo', 'JS']
```

slice: "fotocopia". No modifica el original y el fin NO se incluye.

```
const a = ['A', 'B', 'C', 'D'] const parte = a.slice(1, 3) // ['B','C'] // a sigue intacto
```

Ejercicio de tienda: para quitar la primera aparicion de un elemento del carrito, busca su posicion con indexOf y despues usa splice.

```
const pos = carrito.indexOf('Zelda') if (pos !== -1) { carrito.splice(pos, 1) }
```

E. BUSQUEDA Y VALIDACION

Arrays simples: indexOf/lastIndexOf/includes trabajan directamente con valores. Arrays de objetos: find/findIndex/findLastIndex reciben una condicion.

```
const nombres = ['Ana', 'Luis', 'Ana'] nombres.indexOf('Ana') // 0 nombres.lastIndexOf('Ana') // 2
nombres.includes('Luis') // true
```

```
const usuarios = [ { id: 1, nombre: 'Ana' }, { id: 2, nombre: 'Luis' } ] usuarios.find(u => u.id === 2) // { id: 2,
nombre: 'Luis' } usuarios.findIndex(u => u.id === 2) // 1
```

some: responde "hay al menos uno?". every: responde "todos cumplen?".

```
const stock = [0, 3, 5] stock.some(s => s === 0) // true stock.every(s => s >= 0) // true
```

Aplicacion directa al ejercicio: hayAgotados -> some; preciosValidos -> every.

Regla: si necesitas el objeto para seguir calculando algo, find suele ser mejor que includes; si solo quieres un booleano sobre un valor literal, includes es mas directo.

F. TRANSFORMACION: MAP, FILTER Y FOREACH

map mantiene el numero de elementos y cambia lo que representa cada posicion.

```
const precios = [10, 20, 30] const iva = precios.map(p => p * 1.21) // [12.1, 24.2, 36.3]
```

Ejercicio 1 de arrays: playlist -> forEach para imprimir titulo y artista. No necesitas crear otro array porque solo quieres realizar un efecto.

```
canciones.forEach(cancion => { console.log(`${cancion.titulo} - ${cancion.artista}`) })
```

Ejercicio 2 de arrays: primero filtras canciones de mas de 180 s y despues transformas cada una en string.

```
const resultado = canciones .filter(c => c.duracion > 180) .map(c => `${c.titulo} - ${c.duracion}s`)
```

Ejemplo de tienda: agotados = filter para quedarnos con stock 0 + map para obtener solo nombres.

```
const nombres = catalogo .filter(p => p.stock === 0) .map(p => p.nombre) // ['Leche', 'Huevos']
```

ERROR TIPICO: usar forEach cuando el enunciado pide un array resultante. forEach devuelve undefined; map devuelve un array nuevo.

G. CALCULO: REDUCE Y REDUCERIGHT

reduce convierte muchos elementos en un unico resultado. Para el examen piensa en: total, suma, promedio con acumulador, maximo, agrupacion.

```
const numeros = [1, 2, 3, 4] const total = numeros.reduce((acum, n) => acum + n, 0) // 10
```

Valor inicial: en tu nuevo tema se recomienda especificarlo siempre, especialmente porque un array vacio sin valor inicial produce error.

```
[].reduce((a, b) => a + b, 0) // 0
```

Ejercicio 2.2 - valorAlmacen: cada producto aporta precio * stock.

```
const total = catalogo.reduce((acum, producto) => { return acum + producto.precio * producto.stock }, 0)
```

Ejercicio 2.2 - productoMasCaro: reduce compara dos objetos y conserva el de mayor precio.

```
const masCaro = catalogo.reduce((mejor, p) => { if (mejor === undefined || p.precio > mejor.precio) { return p }
return mejor }, undefined)
```

ReduceRight: misma idea, recorriendo desde la derecha. Ejemplo: convertir [a,b,c] en cba.

```
['a', 'b', 'c'].reduceRight((texto, letra) => texto + letra, '') // 'cba'
```

H. ORDENAR, SPLIT, JOIN Y FLAT

sort numerico: siempre usa comparador cuando ordenas numbers.

```
const nums = [10, 2, 15] nums.sort((a, b) => a - b) // [2,10,15] nums.sort((a, b) => b - a) // [15,10,2]
```

Texto: localeCompare. En tus ejercicios, nombresOrdenados usa map + sort + localeCompare.

```
const nombres = catalogo .map(p => p.nombre) .sort((a, b) => a.localeCompare(b))
```

split/join: 4.1 de tienda usa split para romper el texto del pedido; 4.5 usa join para volver a construir el ticket.

```
'Ana|Pan:2;Leche:1'.split('|') // ['Ana', 'Pan:2;Leche:1'] ['Pan x2', 'Leche x1'].join('
') // 'Pan x2
Leche x1'
```

flat: si tienes arrays dentro de arrays y quieres un solo nivel.

```
const lineas = pedidos.map(p => p.lineas) const todas = lineas.flat()
```

Combinacion muy frecuente: map -> flat -> map -> filter -> sort. Eso es exactamente el tipo de cadena que aparece en productosVendidos.

I. METODOS EXTRA DEL NUEVO TEMA 2.2

Metodo	Sintaxis	Uso	Ejemplo / resultado
findLastIndex	arr.findLastIndex(x => cond)	Ultima posicion que cumple una condicion	usuarios.findLastIndex(u => u.nombre === 'Celina') ->
reduceRight	arr.reduceRight((acc,x)=>..., inicial)	Acumular desde la derecha	[a,b,c] -> cba
fill	arr.fill(valor, inicio, fin)	Rellenar una zona del array; muta	[1,2,3,4].fill(9,1,3) -> [1,9,9,4]
flat	arr.flat(profundidad)	Aplanar arrays anidados	[[1,2],[3]] .flat() -> [1,2,3]
flatMap	arr.flatMap(fn)	Map + un nivel de flat en una sola pasada	['hola mundo'].flatMap(s => s.split(' ')) -> ['hola','mundo
toSorted	arr.toSorted(fn)	Ordenar sin mutar (ES2023)	ordenada = original.toSorted((a,b)=>a-b)
Array.isArray	Array.isArray(v)	Comprobar si v es array	Array.isArray([]) -> true

Prioridad para tu examen: los ejercicios que me pasaste trabajan sobre todo map/filter/reduce/find/findIndex/some/every/sort/localeCompare/push/pop/shift/unshift/splice/slice/indexOf/lastIndexOf/includes/join/split/flat. Los metodos extra entran en tu nuevo tema, asi que conviene reconocer su sintaxis aunque no sean el nucleo de 2.2.

J. MUTABILIDAD, COPIAS Y ENCADENAMIENTO

Mutan: push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin. No mutan: slice, concat, map, filter, find, findIndex, includes, reduce, join, flat, some, every.

Regla del ejercicio 2.2: si la funcion no debe tocar el array de entrada y necesitas ordenar, copia primero.

```
const ordenados = [...productos].sort((a, b) => a.precio - b.precio)
```

Objeto nuevo con spread: util para servirPedido, porque cambias stock sin modificar el objeto de entrada.

```
const nuevoProducto = { ...producto, stock: producto.stock - cantidad }
```

Encadenamiento: puedes usar el resultado de un metodo como entrada del siguiente. Lee de izquierda a derecha.

```
const baratos = productos .filter(p => p.precio < 50) .map(p => p.nombre) // primero selecciona; despues transforma
```

Otro patron: map -> flat -> map -> filter -> sort. No lo memorices como una formula; memoriza que cada paso tiene una finalidad.

K. EJERCICIO 2.2 - PARTE 4: PEDIDOS

4.1 parsearPedido - ejemplo completo con el formato real del ejercicio.

```
const texto = 'Ana|Pan:2;Leche:1' const partes = texto.split('|') const cliente = partes[0] const lineas =
partes[1].split(';').map(linea => { const datos = linea.split(':') return { nombre: datos[0], cantidad:
Number(datos[1]) } }) // { cliente: 'Ana', lineas: [ // {nombre:'Pan', cantidad:2}, {nombre:'Leche', cantidad:1} ] }
```

4.2 puedeServirse - "TODOS" -> every. Hay que recorrer pedido.lineas, buscar cada producto y comparar stock con cantidad.

```
return pedido.lineas.every(linea => { const producto = buscarProducto(catalogo, linea.nombre) return producto !==
undefined && producto.stock >= linea.cantidad })
```

4.3 totalPedido - "TOTAL" -> reduce; cada linea aporta precio * cantidad.

```
return pedido.lineas.reduce((total, linea) => { const producto = buscarProducto(catalogo, linea.nombre) return total +
producto.precio * linea.cantidad }, 0)
```

4.4 servirPedido - "catalogo nuevo" -> map; encuentra la linea y devuelve un objeto con spread.

```
return catalogo.map(producto => { const linea = pedido.lineas.find(l => l.nombre === producto.nombre) if (linea ===
undefined) return producto return { ...producto, stock: producto.stock - linea.cantidad } })
```

4.5 generarTicket - cada linea se transforma en texto y despues se unen con salto de linea.

```
return pedido.lineas .map(linea => { const producto = buscarProducto(catalogo, linea.nombre) return `${linea.nombre}
x${linea.cantidad} = ${producto.precio * linea.cantidad}€` }) .join('
')
```

L. EJERCICIO 2.2 - PARTE 5: ARRAYS MUTABLES

5.1 atenderSiguiente - una cola es FIFO: sale el primero -> shift.

```
export const atenderSiguiente = cola => cola.shift()
```

5.2 agregarUrgente - entra por delante -> unshift.

```
export const agregarUrgente = (cola, pedido) => { cola.unshift(pedido) return cola }
```

5.3 agregarAlCarrito - entra por el final -> push. Si hay historial, registra la accion.

```
carrito.push(producto) historial.push({ tipo: 'agregar', producto })
```

5.4 quitarDelCarrito - primero encuentra la posicion y luego splice(pos, 1).

```
const pos = carrito.indexOf(producto) if (pos !== -1) { carrito.splice(pos, 1) historial.push({ tipo: 'quitar',
producto }) }
```

5.5 deshacer - el historial es una pila LIFO: la ultima accion es la primera que se revierte.

```
const accion = historial.pop() // si fue 'agregar' -> quitar el producto // si fue 'quitar' -> volver a insertarlo //
lastIndexOf ayuda si hay varias copias del mismo producto
```

Reconocimiento instantaneo: cola = shift/unshift; pila/historial = pop/push.

M. EJERCICIO 2.2 - PARTE 6: INTEGRACION

6.1 procesarCola combina while + atenderSiguiente + puedeServirse + servirPedido.

```
while (cola.length > 0) { const pedido = atenderSiguiente(cola) if (puedeServirse(catalogo, pedido)) { catalogo = servirPedido(catalogo, pedido) } }
```

6.2 productosVendidos exige varias etapas. Una forma de leer el problema es: sacar lineas -> aplanar -> sacar nombres -> quitar repetidos -> ordenar.

```
const nombres = pedidos .map(p => p.lineas) .flat() .map(linea => linea.nombre) .filter((nombre, i, arr) => arr.indexOf(nombre) === i) .sort((a, b) => a.localeCompare(b))
```

6.3 graficoStock crea una cadena proporcional al stock. La idea es combinar new Array + fill + join.

```
const barras = new Array(stock).fill('■').join('') // stock = 5 -> '■■■■■'
```

Truco: si una funcion de la parte 6 parece demasiado larga, dividela mentalmente en mini pasos y asigna cada verbo a un metodo.

N. PATRONES DE EXAMEN Y ERRORES QUE CUESTAN PUNTOS

Cuando leas...	Piensa en...	Ejemplo
"TODOS"	every	pedido.lineas.every(...)
"ALGUNO / HAY ALGUNO"	some	catalogo.some(p => p.stock === 0)
"BUSCAR PRODUCTO"	find	catalogo.find(p => ...)
"POSICION"	findIndex / indexOf	findIndex para objetos; indexOf para valor
"NOMBRE DE CADA"	map	catalogo.map(p => p.nombre)
"SOLO LOS QUE"	filter	catalogo.filter(p => p.stock > 0)
"TOTAL / SUMA"	reduce	reduce((acc,p)=>acc+p.precio,0)
"ORDEN ALFABETICO"	sort + localeCompare	sort((a,b)=>a.localeCompare(b))
"NO MODIFICAR"	copia con spread/slice	[...array].sort(...)
"COLA"	shift / unshift	sale el primero
"HISTORIAL / DESHACER"	pop / push	sale la ultima accion
"QUITAR UNO"	indexOf + splice	splice(pos,1)
"TEXTO SEPARADO"	split	texto.split(';')
"UNIR LINEAS"	join	lineas.join("\n")

12 errores que debes revisar antes de entregar

1. Usar = en vez de ===.
2. Recorrer catalogo cuando el enunciado dice "todos los productos del pedido"; debes recorrer pedido.lineas.
3. Comparar producto.stock > 0 cuando la cantidad solicitada puede ser mayor: debe ser stock >= cantidad.
4. Confundir find con findIndex.
5. Usar map para filtrar.
6. Usar forEach esperando un array nuevo.
7. Ordenar numeros con sort() sin comparador.
8. Ordenar la entrada directamente cuando la funcion debe ser inmutable.
9. Olvidar Number() al convertir cantidades que vienen de split().
10. Confundir split (string -> array) con join (array -> string).
11. Olvidar el valor inicial de reduce cuando conviene o cuando el array puede estar vacio.
12. Usar delete en arrays: deja un hueco y no reajusta length; para quitar posiciones usa splice.

CHECKLIST DE 60 SEGUNDOS: 1) ¿He recorrido el array correcto? 2) ¿El metodo coincide con el verbo del enunciado? 3) ¿Tengo que devolver array, elemento, indice, booleano o un unico valor? 4) ¿Estoy mutando el original sin querer? 5) ¿Necesito ===? 6) ¿He convertido con Number? 7) ¿He puesto return donde toca? 8) ¿En sort de numbers he usado a-b/b-a? 9) ¿En texto he usado localeCompare? 10) ¿He protegido find frente a undefined?

FICHA FINAL: LECTURA DE CUALQUIER ENUNCIADO

Ejemplo: "Devuelve los nombres de los productos agotados ordenados alfabeticamente, sin modificar el catalogo".

```
const resultado = catalogo .filter(p => p.stock === 0) .map(p => p.nombre) .sort((a, b) => a.localeCompare(b))
```

Traduccion: agotados -> filter; nombres -> map; alfabeticamente -> sort + localeCompare. Como se pide no modificar el catalogo, el sort se aplica al array nuevo producido por map.

Ejemplo: "Comprueba que todos los productos pedidos existen y tienen stock suficiente".

```
return pedido.lineas.every(linea => { const producto = buscarProducto(catalogo, linea.nombre) return producto !== undefined && producto.stock >= linea.cantidad })
```

Traduccion: todos -> every; producto -> buscarProducto/find; existe -> !== undefined; stock suficiente -> stock >= cantidad.

Ejemplo: "Calcula el valor total del almacen".

```
return catalogo.reduce((total, p) => total + p.precio * p.stock, 0)
```

Traduccion: total -> reduce; valor de cada elemento -> precio * stock; valor inicial -> 0.

Miku: si dudas en el examen, vuelve siempre a esta pregunta: "Que pregunta del enunciado tengo que contestar?" La respuesta suele decirte el metodo.

O. EJEMPLOS DE LOS EJERCICIOS DE ARRAYS QUE YA HICIMOS

Playlist: lista de objetos {titulo, artista, duracion}. Si el objetivo es solo imprimir, usa forEach; si quieres una lista nueva, usa filter/map.

```
canciones.forEach(c => console.log(`${c.titulo} - ${c.artista}`))
```

Canciones largas: primero seleccionas las de mas de 180 segundos y despues las conviertes a texto.

```
const largas = canciones .filter(c => c.duracion > 180) .map(c => `${c.titulo} - ${c.duracion}s`) // ejemplo de resultado: ['Song A - 210s', 'Song C - 195s']
```

Biblioteca digital: un array de objetos {id,titulo,autor,paginas}. Para obtener el total de paginas, reduce; para buscar/eliminar, find + findIndex + splice.

```
const paginas = libros.reduce((total, libro) => total + libro.paginas, 0) const pos = libros.findIndex(libro => libro.id === 4) if (pos !== -1) libros.splice(pos, 1)
```

Validar: some/every sirven para preguntas globales sobre el array.

```
const hayLargos = libros.some(l => l.paginas > 500) const todosValidos = libros.every(l => typeof l.paginas === 'number' && l.paginas > 0)
```

Empleados: filter por departamento, reduce para media salarial y sort para ordenar por salario.

```
const delMarketing = empleados.filter(e => e.departamento === 'Marketing') const media = empleados.reduce((t,e) => t + e.salario, 0) / empleados.length const ordenados = [...empleados].sort((a,b) => b.salario - a.salario)
```

Lo que debes ver: en un mismo ejercicio se combinan metodos; cada uno hace una sola tarea y el siguiente recibe su resultado.

P. EJERCICIO 2.2 - PARTES 1 A 3: PATRONES QUE YA CONTROLAS

Apartado 2.2	Que pide	Patron	Salida/idea
1.1 crearCatalogo	Matriz -> objetos	map	[{nombre,categoria,precio,stock}, ...]
1.2 ampliarCatalogo	Catalogo nuevo + altas	concat	array nuevo
1.3 nombresOrdenados	Nombres alfabeticos	map + sort + localeCompare	[Ana,Zelda]
1.4 ordenarPorPrecio	Copia + orden precio	[...array].sort((a,b)=>...)	orden asc/desc
1.5 tresMasBaratos	3 menores	slice(0,3) + map	3 nombres
2.1 buscarProducto	Un producto, sin importar mayusculas	find + toLowerCase	objeto/undefined
2.2 existeProducto	Booleano usando map/includes	map + includes	true/false
2.3 posicionProducto	Indice de producto	findIndex	indice/-1
2.4 agotados	Nombres stock 0	filter + map	array de nombres
2.5 productosEntre	Precio dentro de rango	filter	array nuevo
3.1 valorAlmacen	Suma precio*stock	reduce	numero
3.2 productoMasCaro	Objeto con precio maximo	reduce	objeto
3.3 unidadesPorCategoria	Agrupar/contar	reduce + acumulador objeto	{cat: unidades,...}
3.4 hayAgotados	Existe stock 0	some	true/false
3.5 preciosValidos	Todos number y >0	every + typeof	true/false

Ejemplo 1.1: si la matriz contiene [Pan,Alimentacion,2,5], map debe producir { nombre:Pan, categoria:Alimentacion, precio:2, stock:5 }.

```
matriz.map(fila => ({ nombre: fila[0], categoria: fila[1], precio: fila[2], stock: fila[3] }))
```

Ejemplo 3.3: cuando reduce construye un objeto, piensa en "si no existe la categoria, inicializala; despues suma las unidades".

```
catalogo.reduce((acc, p) => { if (acc[p.categoria] === undefined) acc[p.categoria] = 0 acc[p.categoria] += p.stock return acc }, {})
```

Conexion con 4.2: both use every, but the array changes. En 3.5 recorres catalogo porque la pregunta habla de todos los precios del catalogo; en 4.2 recorres pedido.lineas porque la pregunta habla de todos los productos del pedido.

Q. FICHA DE CONSTRUCCION: DE DATOS A FUNCION

Paso 1 - Identifica la entrada: array, objeto, string, numero o varios parametros.

```
const funcion = (entrada) => { // ... }
```

Paso 2 - Decide la salida: array nuevo, objeto, elemento, indice, booleano, numero o string.

Salida	Metodos/patrones frecuentes	Ejemplo
Array nuevo	map/filter/slice/concat/flat	productos.map(p=>p.nombre)
Objeto/elemento	find/reduce	catalogo.find(p=>p.id===3)
Indice	findIndex/indexOf	carrito.indexOf(Zelda)
Booleano	some/every/includes	catalogo.some(p=>p.stock===0)
Numero	reduce	catalogo.reduce((t,p)=>t+p.precio,0)
String	join/template literal	lineas.join(\n)

Paso 3 - Pregunta sobre mutabilidad: ¿la funcion puede cambiar la entrada? En 2.2, salvo la Parte 5, las funciones no deben modificar los arrays de entrada.

Paso 4 - Haz la condicion exacta: usa === para igualdad, !== para desigualdad, >= cuando "suficiente" incluye el limite, && cuando deben cumplirse dos cosas.

```
producto !== undefined && producto.stock >= linea.cantidad
```

Paso 5 - Prueba mentalmente un caso: producto inexistente; stock 0; cantidad igual al stock; cantidad mayor al stock; array vacio; repetidos; dos elementos con el mismo nombre.

Resultado mental de algunos patrones:

```
[1,2,3].map(n => n*2) -> [2,4,6] [1,2,3].filter(n => n > 1) -> [2,3] [1,2,3].find(n => n > 1) -> 2 [1,2,3].findIndex(n => n > 1) -> 1 [1,2,3].some(n => n > 2) -> true [1,2,3].every(n => n > 0) -> true [1,2,3].reduce((t,n) => t+n,0) -> 6
```

ANEXO - FUNCION POR FUNCION

JavaScript 2º DAW - Sintaxis + ejemplos + terminal + ejercicios + combinaciones

Este anexo se añade al ultimo documento sin sustituir ni modificar sus paginas originales. Su objetivo es bajar toda la teoria a patrones concretos de examen, sobre todo 4-6 del ejercicio 2.2.

Basado en: tus apuntes de Valores y variables, Estructuras de control, Funciones y programacion funcional, Arrays y Metodos de arrays, ademas del ejercicio 2.2 de tienda.js.

INDICE DEL ANEXO

Bloque	Contenido
A	Como pensar un ejercicio: entrada -> proceso -> salida
B	Crear funciones, funciones flecha, arrays, objetos y arrays de objetos
C	Metodos de arrays: ficha completa metodo por metodo
D	Funciones del ejercicio 2.2: Partes 1-3
E	Funciones del ejercicio 2.2: Parte 4 - Pedidos
F	Funciones del ejercicio 2.2: Parte 5 - Mutacion
G	Funciones del ejercicio 2.2: Parte 6 - Integracion
H	Combinaciones de metodos que debes reconocer al instante
I	Checklist final y patrones de terminal

Regla de lectura: cada ficha te dice que problema resuelve el metodo o la funcion, la sintaxis, un ejemplo real, que sale por consola, en que ejercicios aparece y con que otras piezas suele combinarse.

A. COMO PENSAR CUALQUIER EJERCICIO

Antes de escribir codigo, identifica cuatro cosas:

1. Entrada: que reciben los parametros (array, objeto, string, pedido...).

2. Salida: que debe devolver la funcion (array, objeto, numero, booleano, string...).

3. Accion: transformar, filtrar, buscar, comprobar, acumular, ordenar, mutar...

4. Inmutabilidad: si la funcion no debe modificar el original, usa copias (`[...array]`, `slice()` y `{...objeto}`).

Mapa mental para el examen

```
TRANSFORMAR cada elemento   -> map
QUEDARME con algunos      -> filter
ENCONTRAR uno              -> find
ENCONTRAR posicion         -> findIndex
¿ALGUNO cumple?            -> some
¿TODOS cumplen?            -> every
CALCULAR un total/valor    -> reduce
ORDENAR                    -> sort
BUSCAR valor               -> includes / indexOf
AÑADIR al final            -> push
QUITAR al final            -> pop
AÑADIR al principio        -> unshift
QUITAR al principio        -> shift
QUITAR por posicion        -> splice
COPIAR parte               -> slice
ARRAY -> texto              -> join
TEXTO -> array              -> split
APLANAR                     -> flat
CAMBIAR objeto sin mutar   -> {...objeto, propiedad: nuevoValor}
```

Pregunta estrella: "¿Que tiene que devolver la funcion?". Si la respuesta es "un array nuevo", sospecha de map/filter/slice/concat/flat. Si es "un unico valor", piensa en reduce/find/some/every/indexOf.

B. CONSTRUIR LAS PIEZAS BASICAS

Funciones

```
const sumar = (a, b) => a + b

const resultado = sumar(2, 3)
console.log(resultado) // 5
```

Regla: flecha sin llaves = return implicito; flecha con llaves = necesitas `return`.

Arrays

```
const juegos = ["Zelda", "Mario", "Persona"]
console.log(juegos[0])
console.log(juegos.length)
```

Resultado: Zelda y 3. Los indices empiezan en 0.

Array de objetos

```
const catalogo = [
  { nombre: "Zelda", precio: 60, stock: 5 },
  { nombre: "Mario", precio: 50, stock: 2 }
]

console.log(catalogo[0].nombre)
```

Resultado: Zelda.

Objeto

```
const producto = { nombre: "Zelda", precio: 60 }
producto.precio = 55
console.log(producto.precio)
```

Resultado: 55.

Desestructuracion y spread

```
const { nombre, precio } = producto
const copia = { ...producto, stock: 4 }
console.log(nombre, copia)
```

Resultado: Zelda y un objeto nuevo con stock 4.

JSON

```
const texto = JSON.stringify(producto)
const objeto = JSON.parse(texto)
```

Idea: stringify convierte objeto JS -> texto JSON; parse hace JSON -> objeto JS.

C. METODOS DE ARRAYS - FICHA POR FICHA

forEach()

Tipo: Recorrer / efecto secundario

Idea clave: Cuando quieres ejecutar una accion por cada elemento y no necesitas un nuevo array. En tus apuntes devuelve `undefined`.

Sintaxis

```
arr.forEach((elemento, indice, array) => { ... })
```

Cuando usarlo

Cuando quieres ejecutar una accion por cada elemento y no necesitas un nuevo array. En tus apuntes devuelve `undefined`.

Ejemplo

```
const juegos = ["Zelda", "Mario"]
juegos.forEach((juego, i) => console.log(`${i}: ${juego}`))
```

Resultado en terminal / valor devuelto

0: Zelda 1: Mario

Ejercicios donde aparece

Practica 2.1 - playlist: imprimir titulo + artista.

Cuando se usa junto con otros

Suele ir solo para mostrar o efectuar una accion. Si necesitas resultados en un array, cambia a `map`.

Errores tipicos

`forEach` no devuelve un array. No hagas `const x = arr.forEach(...)` esperando resultados.

Consejo de Miku

Miku: "forEach = haz algo con cada elemento; map = construye un array nuevo."

map()

Tipo: Transformar	**Idea clave:** Cuando quieres una salida por cada entrada: mismo numero de elementos, pero transformados.

Sintaxis

```
arr.map((elemento, indice, array) => expresion)
```

Cuando usarlo

Cuando quieres una salida por cada entrada: mismo numero de elementos, pero transformados.

Ejemplo

```
const precios = [10, 20, 30]
const iva = precios.map(p => p * 1.21)
console.log(iva)
```

Resultado en terminal / valor devuelto

[12.1, 24.2, 36.3]

Ejercicios donde aparece

2.1 - playlist (pasar objetos a strings); 2.2 - crearCatalogo; nombresOrdenados; agotados; generarTicket; varios apartados 6.

Cuando se usa junto con otros

Muy comun despues de `filter`, y dentro de `flatMap`. Tambien aparece despues de buscar datos con `find`.

Errores tipicos

`map` no es para quedarte con algunos: no filtra. Debe devolver el nuevo valor de cada elemento.

Consejo de Miku

Miku: "Si entran 10 elementos, map normalmente produce 10 resultados."

filter()

Tipo: Seleccionar	**Idea clave:** Cuando quieres conservar solamente los elementos que cumplen una condicion. Devuelve un array nuevo.

Sintaxis

```
arr.filter((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando quieres conservar solamente los elementos que cumplen una condicion. Devuelve un array nuevo.

Ejemplo

```
const stock = [{n:"A",s:2},{n:"B",s:0}]
const disponibles = stock.filter(p => p.s > 0)
console.log(disponibles)
```

Resultado en terminal / valor devuelto

[{ n: "A", s: 2 }]

Ejercicios donde aparece

2.1 - canciones >180s; biblioteca; empleados por departamento; 2.2 - agotados; productosEntre.

Cuando se usa junto con otros

Clasico: `filter(...).map(...)` para primero seleccionar y luego transformar.

Errores tipicos

`filter` necesita una condicion. Si devuelves el objeto o un numero accidentalmente, JavaScript lo interpreta como truthy/falsy, pero no estas expresando bien la intencion.

Consejo de Miku

Miku: "filter decide quien se queda; map decide en que se convierte."

find()

Tipo: Busqueda de un elemento	**Idea clave:** Cuando buscas el primer elemento que cumple una condicion. Ideal para arrays de objetos. Si no hay ninguno devuelve `undefined`.

Sintaxis

```
arr.find((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando buscas el primer elemento que cumple una condicion. Ideal para arrays de objetos. Si no hay ninguno devuelve `undefined`.

Ejemplo

```
const usuarios = [{id:1,nombre:"Celina"},{id:2,nombre:"David"}]
console.log(usuarios.find(u => u.id === 2))
```

Resultado en terminal / valor devuelto

{ id: 2, nombre: "David" }

Ejercicios donde aparece

2.1 - buscar libros; 2.2 - buscarProducto; puedeServirse; totalPedido; servirPedido.

Cuando se usa junto con otros

Muy habitual: `const producto = buscarProducto(...); producto !== undefined` y luego leer `producto.stock`.

Errores tipicos

No confundas `find` con `filter`: uno devuelve un elemento, el otro un array.

Consejo de Miku

Miku: "Si el enunciado dice un producto/usuario/libro, piensa find."

findIndex()

Tipo: Busqueda de posicion

Idea clave: Cuando necesitas el indice del primer elemento que cumple la condicion. Devuelve `-1` si no encuentra.

Sintaxis

```
arr.findIndex((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando necesitas el indice del primer elemento que cumple la condicion. Devuelve `-1` si no encuentra.

Ejemplo

```
const juegos = [{n:"Zelda"},{n:"Mario"}]
console.log(juegos.findIndex(j => j.n === "Mario"))
```

Resultado en terminal / valor devuelto

1

Ejercicios donde aparece

2.1 - buscar y borrar libros; 2.2 - posicionProducto.

Cuando se usa junto con otros

Patron clasico para borrar: `const i = arr.findIndex(...); arr.splice(i, 1)`.

Errores tipicos

Si devuelve `-1`, no deberias hacer `splice(-1,1)` sin comprobarlo.

Consejo de Miku

Miku: "findIndex = necesito la puerta; find = necesito el personaje."

findLastIndex()

Tipo: Busqueda de posicion desde el final	**Idea clave:** Cuando necesitas la posicion del ultimo elemento que cumple una condicion, empezando por la derecha.

Sintaxis

```
arr.findLastIndex((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando necesitas la posicion del ultimo elemento que cumple una condicion, empezando por la derecha.

Ejemplo

```
const u = [{n:"Celina"},{n:"David"},{n:"Celina"}]
console.log(u.findLastIndex(x => x.n === "Celina"))
```

Resultado en terminal / valor devuelto

2

Ejercicios donde aparece

UT 2.2 - metodo adicional de apuntes; util para historial o ultimas apariciones.

Cuando se usa junto con otros

Puede sustituir a `lastIndexOf` cuando no buscas un valor exacto sino una condicion en objetos.

Errores tipicos

No es lo mismo que `findIndex`: uno busca desde el principio y el otro desde el final.

Consejo de Miku

Miku: "Es el hermano de findIndex que empieza por la derecha."

indexOf()

Tipo: Busqueda exacta por posicion **Idea clave:** Cuando buscas la primera aparicion exacta de un valor. Devuelve indice o `-1`.

Sintaxis

```
arr.indexOf(elemento, desdeOpcional)
```

Cuando usarlo

Cuando buscas la primera aparicion exacta de un valor. Devuelve indice o `-1`.

Ejemplo

```
const a = ["Pan","Leche","Pan"]
console.log(a.indexOf("Pan"))
```

Resultado en terminal / valor devuelto

0

Ejercicios donde aparece

2.2 - existeProducto (a traves del ejercicio); quitarDelCarrito; productosVendidos para quitar duplicados.

Cuando se usa junto con otros

Se combina con `splice` para borrar una primera aparicion y con `filter` para detectar duplicados con `indexOf`.

Errores tipicos

No sirve para buscar por propiedad dentro de objetos. Para eso usa `find`/`findIndex`.

Consejo de Miku

Miku: "Si tengo un string o numero exacto, indexOf. Si tengo una regla, findIndex."

lastIndexOf()

Tipo: Busqueda exacta por la ultima posicion **Idea clave:** Cuando necesitas la ultima aparicion exacta de un valor.

Sintaxis

```
arr.lastIndexOf(elemento, desdeOpcional)
```

Cuando usarlo

Cuando necesitas la ultima aparicion exacta de un valor.

Ejemplo

```
const a = ["Pan","Leche","Pan"]
console.log(a.lastIndexOf("Pan"))
```

Resultado en terminal / valor devuelto

2

Ejercicios donde aparece

2.2 - deshacer, para localizar la ultima aparicion al revertir acciones.

Cuando se usa junto con otros

Puede ir con `splice` para quitar la ultima aparicion.

Errores tipicos

No confundir con `findLastIndex`: lastIndexOf busca igualdad del valor; findLastIndex aplica una funcion.

Consejo de Miku

Miku: "right to left = lastIndexOf/findLastIndex."

includes()

Tipo: Busqueda booleana **Idea clave:** Cuando solo importa si el valor existe: devuelve `true` o `false`.

Sintaxis

```
arr.includes(valor, desdeOpcional)
```

Cuando usarlo

Cuando solo importa si el valor existe: devuelve `true` o `false`.

Ejemplo

```
const productos = ["Pan","Leche"]
console.log(productos.includes("Leche"))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

2.2 - existeProducto (el enunciado exige `map` + `includes`); busquedas sencillas de valores.

Cuando se usa junto con otros

Ejemplo tipico: `catalogo.map(p => p.nombre).includes(nombre)`.

Errores tipicos

No devuelve el elemento ni la posicion.

Consejo de Miku

Miku: "includes responde una sola pregunta: esta o no esta?"

some()

Tipo: Comprobacion de existencia por condicion

Idea clave: Cuando quieres saber si **al menos uno** cumple la condicion.

Sintaxis

```
arr.some((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando quieres saber si **al menos uno** cumple la condicion.

Ejemplo

```
const stock = [0, 3, 5]
console.log(stock.some(s => s === 0))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

2.1 - validar longitudes; 2.2 - hayAgotados.

Cuando se usa junto con otros

Suele ir solo; si necesitas todos, cambia a `every`.

Errores tipicos

No significa "todos". En cuanto encuentra uno que cumple, responde `true`.

Consejo de Miku

Miku: "some = uno basta."

every()

Tipo: Comprobacion de todos **Idea clave:** Cuando quieres saber si **todos** cumplen la condicion.

Sintaxis

```
arr.every((elemento, indice, array) => condicion)
```

Cuando usarlo

Cuando quieres saber si **todos** cumplen la condicion.

Ejemplo

```
const stock = [3, 5, 1]
console.log(stock.every(s => s > 0))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

2.1 - validacion de arrays; 2.2 - preciosValidos; puedeServirse.

Cuando se usa junto con otros

En `puedeServirse`: `pedido.lineas.every(...)`. Dentro suele aparecer `find`/`buscarProducto`.

Errores tipicos

No recorras el array equivocado: en 4.2 se recorre `pedido.lineas`, no `catalogo`.

Consejo de Miku

Miku: "La palabra TODOS del enunciado es una alarma para every."

reduce()

Tipo: Calculo / acumulacion	**Idea clave:** Cuando conviertes muchos elementos en un solo resultado: suma, promedio, valor de inventario, agrupacion...

Sintaxis

```
arr.reduce((acumulador, elemento, indice, array) => nuevoAcumulador, valorInicial)
```

Cuando usarlo

Cuando conviertes muchos elementos en un solo resultado: suma, promedio, valor de inventario, agrupacion...

Ejemplo

```
const numeros = [1,2,3,4]
const total = numeros.reduce((s, n) => s + n, 0)
console.log(total)
```

Resultado en terminal / valor devuelto

10

Ejercicios donde aparece

2.1 - total paginas; salario medio; 2.2 - valorAlmacen; unidadesPorCategoria; totalPedido.

Cuando se usa junto con otros

Muy habitual con `find` dentro para buscar datos antes de acumular: `reduce(... { const p = buscarProducto(...) ...})`.

Errores tipicos

Olvidar el valor inicial en un array vacio puede lanzar un error. En tus apuntes se recomienda especificarlo.

Consejo de Miku

Miku: "reduce empieza con un acumulador y lo actualiza hasta dejar un solo resultado."

reduceRight()

Tipo: Acumulacion de derecha a izquierda

Idea clave: Cuando el orden importa y quieres acumular empezando por el final.

Sintaxis

```
arr.reduceRight((acumulador, elemento, indice, array) => ..., valorInicial)
```

Cuando usarlo

Cuando el orden importa y quieres acumular empezando por el final.

Ejemplo

```
const letras = ["a","b","c"]
console.log(letras.reduceRight((t, x) => t + x, ""))
```

Resultado en terminal / valor devuelto

cba

Ejercicios donde aparece

UT 2.2 - metodo adicional del apunte.

Cuando se usa junto con otros

Similar a `reduce`, pero invierte la direccion de procesamiento. No suele ser central en tienda.js 2.2.

Errores tipicos

No lo uses solo porque "suena mas avanzado"; si el resultado normal sirve, usa `reduce`.

Consejo de Miku

Miku: "Right = derecha."

sort()

Tipo: Ordenacion	**Idea clave:** Cuando necesitas ordenar un array. Con numeros usa comparador; con textos usa `localeCompare`.

Sintaxis

```
arr.sort((a, b) => a - b)
```

Cuando usarlo

Cuando necesitas ordenar un array. Con numeros usa comparador; con textos usa `localeCompare`.

Ejemplo

```
const n = [10, 9, 1, 25]
n.sort((a,b) => a-b)
console.log(n)
```

Resultado en terminal / valor devuelto

[1, 9, 10, 25]

Ejercicios donde aparece

2.1 - ordenar libros/empleados; 2.2 - nombresOrdenados, ordenarPorPrecio, tresMasBaratos, productosVendidos.

Cuando se usa junto con otros

Si no debes mutar, copia antes: `[...arr].sort(...)`. `sort` + `reverse` aparece en ordenarPorPrecio.

Errores tipicos

Sin comparador, los numeros se ordenan como texto. Ademas `sort` muta.

Consejo de Miku

Miku: "sort sabe ordenar, pero no sabe que tus numeros son numeros hasta que le das el comparador."

localeCompare()

Tipo: Ordenacion de textos	**Idea clave:** Para comparar cadenas respetando reglas del idioma. Ideal para alfabetico con acentos.

Sintaxis

```
a.localeCompare(b)
```

Cuando usarlo

Para comparar cadenas respetando reglas del idioma. Ideal para alfabetico con acentos.

Ejemplo

```
const n = ["Zoe","Álvaro","Ana"]
n.sort((a,b) => a.localeCompare(b))
console.log(n)
```

Resultado en terminal / valor devuelto

["Álvaro", "Ana", "Zoe"]

Ejercicios donde aparece

2.2 - nombresOrdenados y productosVendidos.

Cuando se usa junto con otros

Se usa dentro de `sort`: `array.sort((a,b) => a.propiedad.localeCompare(b.propiedad))`.

Errores tipicos

No pongas solo `sort()` si el requisito es alfabetico fiable con acentos.

Consejo de Miku

Miku: "Texto con idioma -> localeCompare."

reverse()

Tipo: Invertir orden **Idea clave:** Cuando quieres invertir el orden actual.

Sintaxis

```
arr.reverse()
```

Cuando usarlo

Cuando quieres invertir el orden actual.

Ejemplo

```
const n = [1,2,3]
n.reverse()
console.log(n)
```

Resultado en terminal / valor devuelto

[3, 2, 1]

Ejercicios donde aparece

2.2 - ordenarPorPrecio en el requisito de descendente.

Cuando se usa junto con otros

Puede seguir a `sort((a,b)=>a-b)` si quieres ascendente primero y despues invertir.

Errores tipicos

Modifica el original. Si no debes mutar, usa `[...arr].reverse()`.

Consejo de Miku

Miku: "reverse no ordena por valor; solo da la vuelta."

push()

Tipo: Añadir al final	**Idea clave:** Para añadir uno o varios elementos al final del array. Muta el original y devuelve la nueva longitud.

Sintaxis

```
arr.push(...elementos)
```

Cuando usarlo

Para añadir uno o varios elementos al final del array. Muta el original y devuelve la nueva longitud.

Ejemplo

```
const c = ["A","B"]
const r = c.push("C")
console.log(c, r)
```

Resultado en terminal / valor devuelto

["A", "B", "C"] y 3

Ejercicios donde aparece

2.1 - biblioteca/empleados; 2.2 - agregarAlCarrito, cola (entrada).

Cuando se usa junto con otros

Cola FIFO tipica: `push` entra y `shift` sale.

Errores tipicos

No confundas lo que devuelve: `push` devuelve la longitud, no el elemento.

Consejo de Miku

Miku: "push = entra por atras."

pop()

Tipo: Quitar del final **Idea clave:** Para quitar y devolver el ultimo elemento. Muta.

Sintaxis

```
arr.pop()
```

Cuando usarlo

Para quitar y devolver el ultimo elemento. Muta.

Ejemplo

```
const p = ["A","B"]
const x = p.pop()
console.log(p, x)
```

Resultado en terminal / valor devuelto

["A"] y "B"

Ejercicios donde aparece

2.2 - deshacer; pila/historial.

Cuando se usa junto con otros

Pila LIFO: `push` para meter, `pop` para sacar. En `deshacer`, `historial.pop()` obtiene la ultima accion.

Errores tipicos

En array vacio devuelve `undefined`.

Consejo de Miku

Miku: "pop = sale el ultimo."

shift()

Tipo: Quitar del principio **Idea clave:** Para quitar y devolver el primer elemento. Muta.

Sintaxis

```
arr.shift()
```

Cuando usarlo

Para quitar y devolver el primer elemento. Muta.

Ejemplo

```
const cola = ["P1","P2"]
const x = cola.shift()
console.log(cola, x)
```

Resultado en terminal / valor devuelto

["P2"] y "P1"

Ejercicios donde aparece

2.2 - atenderSiguiente, procesarCola.

Cuando se usa junto con otros

Cola FIFO: `push` + `shift`.

Errores tipicos

No uses `pop` en una cola si el requisito es FIFO.

Consejo de Miku

Miku: "shift = sale el primero."

unshift()

Tipo: Añadir al principio **Idea clave:** Para insertar uno o varios elementos al principio. Muta y devuelve nueva longitud.

Sintaxis

```
arr.unshift(...elementos)
```

Cuando usarlo

Para insertar uno o varios elementos al principio. Muta y devuelve nueva longitud.

Ejemplo

```
const q = ["Normal"]
const r = q.unshift("URGENTE")
console.log(q, r)
```

Resultado en terminal / valor devuelto

["URGENTE", "Normal"] y 2

Ejercicios donde aparece

2.2 - agregarUrgente.

Cuando se usa junto con otros

Cola con prioridad: `unshift` pone delante; despues `shift` atiende al primero.

Errores tipicos

Devuelve longitud, no el elemento insertado.

Consejo de Miku

Miku: "unshift = mete por delante."

splice()

Tipo: Insertar / borrar / reemplazar **Idea clave:** Cuando necesitas modificar el array en una posicion concreta.

Sintaxis

```
arr.splice(indice, cuantosBorrar, ...elementos)
```

Cuando usarlo

Cuando necesitas modificar el array en una posicion concreta.

Ejemplo

```
const a = ["A","B","C"]
a.splice(1,1,"X")
console.log(a)
```

Resultado en terminal / valor devuelto

["A", "X", "C"]

Ejercicios donde aparece

2.1 - biblioteca eliminar; 2.2 - quitarDelCarrito, deshacer y operaciones de parte 5.

Cuando se usa junto con otros

Patron de examen: `indexOf` o `findIndex` -> obtienes posicion -> `splice(posicion, 1)`.

Errores tipicos

Modificaras el original. `delete arr[i]` deja un hueco y no desplaza elementos.

Consejo de Miku

Miku: "splice es la navaja suiza del array: borra, inserta y reemplaza."

slice()

Tipo: Copiar / extraer parte	**Idea clave:** Para obtener una copia de una parte sin modificar el original. El final no se incluye.

Sintaxis

```
arr.slice(inicio, fin)
```

Cuando usarlo

Para obtener una copia de una parte sin modificar el original. El final no se incluye.

Ejemplo

```
const a = ["A","B","C","D"]
console.log(a.slice(1,3))
```

Resultado en terminal / valor devuelto

["B", "C"]

Ejercicios donde aparece

2.2 - tresMasBaratos; copiar array completo; evitar mutacion.

Cuando se usa junto con otros

Puede sustituir a `[...arr]` para copiar y luego `sort`.

Errores tipicos

Confundir `slice` y `splice`.

Consejo de Miku

Miku: "slice = fotocopia; splice = cirugia."

concat()

Tipo: Combinar arrays **Idea clave:** Para producir un nuevo array combinado sin mutar los originales.

Sintaxis

```
arr.concat(...arraysOValores)
```

Cuando usarlo

Para producir un nuevo array combinado sin mutar los originales.

Ejemplo

```
const a = [1,2]
const b = [3,4]
console.log(a.concat(b))
```

Resultado en terminal / valor devuelto

[1, 2, 3, 4]

Ejercicios donde aparece

2.1 - ampliarCatalogo; util para unir resultados.

Cuando se usa junto con otros

Alternativa ES6: `[...a, ...b]`.

Errores tipicos

No esperes que modifique el array original.

Consejo de Miku

Miku: "concat une; push mete dentro del original."

split()

Tipo: String -> array **Idea clave:** Cuando una cadena codifica varios datos separados por un caracter.

Sintaxis

```
texto.split(separador, limiteOpcional)
```

Cuando usarlo

Cuando una cadena codifica varios datos separados por un caracter.

Ejemplo

```
const texto = "Ana|Pan:2;Leche:1"
console.log(texto.split("|"))
```

Resultado en terminal / valor devuelto

["Ana", "Pan:2;Leche:1"]

Ejercicios donde aparece

2.2 - parsearPedido (split por `|`, `;` y `:`).

Cuando se usa junto con otros

En 4.1 se encadenan `split("|")` -> `split(";")` -> `split(":")`.

Errores tipicos

No confundas con `join`, que hace el viaje contrario.

Consejo de Miku

Miku: "split corta una cadena en trozos."

join()

Tipo: Array -> string **Idea clave:** Cuando necesitas unir todos los elementos en un unico texto.

Sintaxis

```
arr.join(separador)
```

Cuando usarlo

Cuando necesitas unir todos los elementos en un unico texto.

Ejemplo

```
const a = ["Pan","Leche","Huevos"]
console.log(a.join(" - "))
```

Resultado en terminal / valor devuelto

Pan - Leche - Huevos

Ejercicios donde aparece

2.2 - generarTicket; 6.3 graficoStock; ejemplos de arrays a texto.

Cuando se usa junto con otros

Muy comun despues de `map`: `array.map(...).join("\n")` para construir lineas.

Errores tipicos

No lo confundas con `concat`; `join` produce string.

Consejo de Miku

Miku: "join cose un array en una sola cadena."

flat()

Tipo: Aplanar arrays **Idea clave:** Cuando tienes arrays anidados y quieres reducir niveles.

Sintaxis

```
arr.flat(profundidad)
```

Cuando usarlo

Cuando tienes arrays anidados y quieres reducir niveles.

Ejemplo

```
const a = [["A","B"],["C"]]
console.log(a.flat())
```

Resultado en terminal / valor devuelto

["A", "B", "C"]

Ejercicios donde aparece

2.2 - productosVendidos; UT 2.2 - ejemplo de apuntes.

Cuando se usa junto con otros

Patron tipico: `pedidos.map(...).flat()` para juntar lineas de varios pedidos.

Errores tipicos

Con demasiados niveles, `flat()` por defecto solo baja un nivel; usa una profundidad adecuada.

Consejo de Miku

Miku: "flat quita una capa de cajas."

flatMap()

Tipo: Map + flat de un nivel	**Idea clave:** Cuando cada elemento produce cero, uno o varios elementos y quieres aplanarlo inmediatamente.

Sintaxis

```
arr.flatMap(elemento => arrayDeResultados)
```

Cuando usarlo

Cuando cada elemento produce cero, uno o varios elementos y quieres aplanarlo inmediatamente.

Ejemplo

```
const a = ["hola mundo"]
console.log(a.flatMap(x => x.split(" ")))
```

Resultado en terminal / valor devuelto

["hola", "mundo"]

Ejercicios donde aparece

UT 2.2 - metodo adicional del apunte; util en transformaciones anidadas.

Cuando se usa junto con otros

Puede sustituir `map(...).flat()` en algunos casos.

Errores tipicos

No aplana mas de un nivel con la misma llamada.

Consejo de Miku

Miku: "flatMap hace dos pasos juntos cuando el resultado encaja."

fill()

Tipo: Rellenar **Idea clave:** Para rellenar posiciones con el mismo valor. Muta el array.

Sintaxis

```
arr.fill(valor, inicioOpcional, finOpcional)
```

Cuando usarlo

Para rellenar posiciones con el mismo valor. Muta el array.

Ejemplo

```
const a = new Array(3).fill("X")
console.log(a)
```

Resultado en terminal / valor devuelto

["X", "X", "X"]

Ejercicios donde aparece

2.2 - graficoStock usa `new Array(n).fill("■")`.

Cuando se usa junto con otros

Muy frecuente en barras o plantillas donde necesitas repetir un caracter.

Errores tipicos

Modifica el array; `new Array(n)` con un solo numero crea longitud, no un elemento con ese numero.

Consejo de Miku

Miku: "new Array(n).fill(x) = prepara n huecos y rellena."

Array.isArray()

Tipo: Comprobacion de tipo **Idea clave:** Para saber si un valor es realmente un array. `typeof []` devuelve `object`.

Sintaxis

```
Array.isArray(valor)
```

Cuando usarlo

Para saber si un valor es realmente un array. `typeof []` devuelve `object`.

Ejemplo

```
console.log(Array.isArray([1,2]))
console.log(Array.isArray({}))
```

Resultado en terminal / valor devuelto

true false

Ejercicios donde aparece

2.2 - crearCatalogo debe comprobar que la entrada es un array; 2.1 - validacion.

Cuando se usa junto con otros

Muy util al principio de una funcion: `if (!Array.isArray(matriz)) return []`.

Errores tipicos

No uses `typeof valor === "array"` porque ese tipo no existe.

Consejo de Miku

Miku: "Array.isArray cuando el enunciado dice si es un array."

toSorted()

Tipo: Ordenacion sin mutar	**Idea clave:** Cuando quieres la version no mutante de sort en entornos modernos ES2023. En tu practica, la via segura pedida es copiar y sort.

Sintaxis

```
arr.toSorted((a,b) => a-b)
```

Cuando usarlo

Cuando quieres la version no mutante de sort en entornos modernos ES2023. En tu practica, la via segura pedida es copiar y sort.

Ejemplo

```
const a=[3,1,2]
const b=a.toSorted((x,y)=>x-y)
console.log(a,b)
```

Resultado en terminal / valor devuelto

[3,1,2] / [1,2,3]

Ejercicios donde aparece

UT 2.2 - metodo adicional de apuntes; en el ejercicio 2.2 usa mejor `[...array].sort(...)` segun sus normas.

Cuando se usa junto con otros

Alternativa mas moderna a `[...array].sort(...)`.

Errores tipicos

Puede no ser la opcion esperada si el examen exige ES6 o el profesor pide expresamente copia con spread/slice.

Consejo de Miku

Miku: "Conoce toSorted, pero en tu ejercicio prioriza la tecnica que te pide el profesor."

D. FUNCIONES DEL EJERCICIO 2.2 - PARTES 1 A 3

1.1 crearCatalogo

Tipo: Construccion

Idea clave: Convierte filas `[nombre,categoria,precio,stock]` en objetos. Si la entrada no es array, devuelve `[]`.

Sintaxis

```
export const crearCatalogo = (matriz) => { ... }
```

Cuando usarlo

Convierte filas `[nombre,categoria,precio,stock]` en objetos. Si la entrada no es array, devuelve `[]`.

Ejemplo

```
const matriz = [["Zelda","Juego",60,3]]
console.log(crearCatalogo(matriz))
```

Resultado en terminal / valor devuelto

[{ nombre: "Zelda", categoria: "Juego", precio: 60, stock: 3 }]

Ejercicios donde aparece

Parte 1.1.

Cuando se usa junto con otros

Array.isArray -> map -> objeto literal.

Errores tipicos

No usar `typeof matriz === "array"`.

Consejo de Miku

Regla: una fila se convierte en un objeto.

1.2 ampliarCatalogo

Tipo: Transformar + combinar **Idea clave:** Crea un catalogo nuevo y añade los productos nuevos sin modificar el original.

Sintaxis

```
export const ampliarCatalogo = (catalogo, matrizNuevos) => { ... }
```

Cuando usarlo

Crea un catalogo nuevo y añade los productos nuevos sin modificar el original.

Ejemplo

```
const base = [{nombre:"A",categoria:"X",precio:1,stock:1}]
const nuevos = [["B","Y",2,3]]
console.log(ampliarCatalogo(base,nuevos))
```

Resultado en terminal / valor devuelto

[A, B] como dos objetos del catalogo

Ejercicios donde aparece

Parte 1.2.

Cuando se usa junto con otros

crearCatalogo + concat.

Errores tipicos

Usar push en el catalogo cuando el ejercicio pide no mutar.

Consejo de Miku

Miku: "Si te dicen nuevo catalogo, sospecha de concat o spread."

1.3 nombresOrdenados

Tipo: Transformar + ordenar texto **Idea clave:** Devuelve solo los nombres y los ordena alfabeticamente respetando acentos.

Sintaxis

```
export const nombresOrdenados = (catalogo) => { ... }
```

Cuando usarlo

Devuelve solo los nombres y los ordena alfabeticamente respetando acentos.

Ejemplo

```
const c=[{nombre:"Zoe"},{nombre:"Álvaro"},{nombre:"Ana"}]
console.log(nombresOrdenados(c))
```

Resultado en terminal / valor devuelto

["Álvaro", "Ana", "Zoe"]

Ejercicios donde aparece

Parte 1.3.

Cuando se usa junto con otros

map(p=>p.nombre) + sort(localeCompare).

Errores tipicos

No ordenar objetos si la salida pedida son strings.

Consejo de Miku

Mapa mental: "nombres" -> map; "ordenados" -> sort.

1.4 ordenarPorPrecio

Tipo: Ordenar copia **Idea clave:** Ordena por precio sin tocar el catalogo original.

Sintaxis

```
export const ordenarPorPrecio = (catalogo, descendente = false) => { ... }
```

Cuando usarlo

Ordena por precio sin tocar el catalogo original.

Ejemplo

```
const c=[{precio:30},{precio:10},{precio:20}]
console.log(ordenarPorPrecio(c))
```

Resultado en terminal / valor devuelto

[{precio:10},{precio:20},{precio:30}]

Ejercicios donde aparece

Parte 1.4.

Cuando se usa junto con otros

[...catalogo].sort((a,b)=>a.precio-b.precio); if descendente, reverse().

Errores tipicos

Ordenar directamente `catalogo.sort` rompe la norma de no mutar.

Consejo de Miku

Miku: "Primero copia, despues ordena."

1.5 tresMasBaratos

Tipo: Seleccionar + ordenar **Idea clave:** Obtiene los nombres de los tres productos mas baratos.

Sintaxis

```
export const tresMasBaratos = (catalogo) => { ... }
```

Cuando usarlo

Obtiene los nombres de los tres productos mas baratos.

Ejemplo

```
const c=[{nombre:"A",precio:30},{nombre:"B",precio:10},{nombre:"C",precio:20},{nombre:"D",precio:5}]
console.log(tresMasBaratos(c))
```

Resultado en terminal / valor devuelto

["D", "B", "C"]

Ejercicios donde aparece

Parte 1.5.

Cuando se usa junto con otros

ordenarPorPrecio + slice(0,3) + map(p=>p.nombre).

Errores tipicos

No uses `splice` porque modificaria.

Consejo de Miku

Patron de encadenamiento: ordenar -> cortar -> transformar.

2.1 buscarProducto

Tipo: Busqueda de objeto **Idea clave:** Busca un producto ignorando mayusculas/minusculas.

Sintaxis

```
export const buscarProducto = (catalogo, nombre) => { ... }
```

Cuando usarlo

Busca un producto ignorando mayusculas/minusculas.

Ejemplo

```
const c=[{nombre:"Zelda",precio:60}]
console.log(buscarProducto(c,"zelda"))
```

Resultado en terminal / valor devuelto

{ nombre: "Zelda", precio: 60 }

Ejercicios donde aparece

Parte 2.1; reaparece en 4.2-4.5.

Cuando se usa junto con otros

find + toLowerCase.

Errores tipicos

Comparar directamente sin normalizar puede fallar con mayusculas.

Consejo de Miku

Miku: "Esta funcion sera una pieza reutilizable para 4 y 6."

2.2 existeProducto

Tipo: Busqueda booleana **Idea clave:** Devuelve true si el nombre aparece. El ejercicio exige map + includes.

Sintaxis

```
export const existeProducto = (catalogo, nombre) => { ... }
```

Cuando usarlo

Devuelve true si el nombre aparece. El ejercicio exige map + includes.

Ejemplo

```
const c=[{nombre:"Zelda"},{nombre:"Mario"}]
console.log(existeProducto(c,"Mario"))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

Parte 2.2.

Cuando se usa junto con otros

map(p=>p.nombre) -> includes(nombre).

Errores tipicos

No sustituir por find si el enunciado dice que map + includes son obligatorios.

Consejo de Miku

Miku: "Aqui el requisito tecnico importa tanto como el resultado."

2.3 posicionProducto

Tipo: Busqueda de indice **Idea clave:** Devuelve el indice del producto o -1.

Sintaxis

```
export const posicionProducto = (catalogo, nombre) => { ... }
```

Cuando usarlo

Devuelve el indice del producto o -1.

Ejemplo

```
const c=[{nombre:"Zelda"},{nombre:"Mario"}]
console.log(posicionProducto(c,"Mario"))
```

Resultado en terminal / valor devuelto

1

Ejercicios donde aparece

Parte 2.3.

Cuando se usa junto con otros

findIndex.

Errores tipicos

No devolver el objeto encontrado.

Consejo de Miku

Miku: "Indice -> findIndex."

2.4 agotados

Tipo: Filtrar + transformar **Idea clave:** Obtiene nombres de productos con stock 0.

Sintaxis

```
export const agotados = (catalogo) => { ... }
```

Cuando usarlo

Obtiene nombres de productos con stock 0.

Ejemplo

```
const c=[{nombre:"A",stock:0},{nombre:"B",stock:2}]
console.log(agotados(c))
```

Resultado en terminal / valor devuelto

["A"]

Ejercicios donde aparece

Parte 2.4.

Cuando se usa junto con otros

filter(p=>p.stock===0) + map(p=>p.nombre).

Errores tipicos

Intentar hacerlo todo con map produce nombres no filtrados.

Consejo de Miku

Selecciona primero, transforma despues.

2.5 productosEntre

Tipo: Filtrar por rango **Idea clave:** Obtiene productos cuyo precio esta dentro del rango inclusivo.

Sintaxis

```
export const productosEntre = (catalogo, minimo, maximo) => { ... }
```

Cuando usarlo

Obtiene productos cuyo precio esta dentro del rango inclusivo.

Ejemplo

```
const c=[{nombre:"A",precio:10},{nombre:"B",precio:30},{nombre:"C",precio:50}]
console.log(productosEntre(c,20,50))
```

Resultado en terminal / valor devuelto

[B, C]

Ejercicios donde aparece

Parte 2.5.

Cuando se usa junto con otros

filter(p=>p.precio>=minimo && p.precio<=maximo).

Errores tipicos

No usar `||`; la condicion del rango necesita `&&`.

Consejo de Miku

Enunciado "entre A y B, incluidos" -> >= y <=.

3.1 valorAlmacen

Tipo: Calculo **Idea clave:** Suma precio x stock de todos los productos.

Sintaxis

```
export const valorAlmacen = (catalogo) => { ... }
```

Cuando usarlo

Suma precio x stock de todos los productos.

Ejemplo

```
const c=[{precio:10,stock:2},{precio:5,stock:3}]
console.log(valorAlmacen(c))
```

Resultado en terminal / valor devuelto

35

Ejercicios donde aparece

Parte 3.1.

Cuando se usa junto con otros

reduce((total,p)=>total+p.precio*p.stock,0).

Errores tipicos

Olvidar multiplicar precio por stock.

Consejo de Miku

Total -> reduce.

3.2 productoMasCaro

Tipo: Maximo por reduce **Idea clave:** Devuelve el objeto con mayor precio.

Sintaxis

```
export const productoMasCaro = (catalogo) => { ... }
```

Cuando usarlo

Devuelve el objeto con mayor precio.

Ejemplo

```
const c=[{nombre:"A",precio:10},{nombre:"B",precio:30}]
console.log(productoMasCaro(c))
```

Resultado en terminal / valor devuelto

{ nombre: "B", precio: 30 }

Ejercicios donde aparece

Parte 3.2.

Cuando se usa junto con otros

reduce comparando acumulador y actual; acumulador inicial undefined.

Errores tipicos

Comparar strings o precios incorrectamente.

Consejo de Miku

Miku: "reduce tambien puede devolver un objeto, no solo numeros."

3.3 unidadesPorCategoria

Tipo: Agrupacion + reduce **Idea clave:** Suma stock agrupado por categoria y devuelve un objeto acumulador.

Sintaxis

```
export const unidadesPorCategoria = (catalogo) => { ... }
```

Cuando usarlo

Suma stock agrupado por categoria y devuelve un objeto acumulador.

Ejemplo

```
const c=[{categoria:"RPG",stock:2},{categoria:"RPG",stock:3},{categoria:"Arcade",stock:1}]
console.log(unidadesPorCategoria(c))
```

Resultado en terminal / valor devuelto

{ RPG: 5, Arcade: 1 }

Ejercicios donde aparece

Parte 3.3.

Cuando se usa junto con otros

reduce con acumulador objeto: acc[categoria] = (acc[categoria] || 0) + stock.

Errores tipicos

Olvidar inicializar la propiedad cuando aparece por primera vez.

Consejo de Miku

Miku: "reduce puede construir estructuras."

3.4 hayAgotados

Tipo: Comprobacion booleana **Idea clave:** Devuelve true si al menos un producto tiene stock 0.

Sintaxis

```
export const hayAgotados = (catalogo) => { ... }
```

Cuando usarlo

Devuelve true si al menos un producto tiene stock 0.

Ejemplo

```
const c=[{stock:2},{stock:0}]
console.log(hayAgotados(c))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

Parte 3.4.

Cuando se usa junto con otros

some(p=>p.stock===0).

Errores tipicos

Usar every: cambiaria "al menos uno" por "todos".

Consejo de Miku

ALGUNO -> some.

3.5 preciosValidos

Tipo: Validacion booleana **Idea clave:** Comprueba que todos los precios son numbers y mayores que 0.

Sintaxis

```
export const preciosValidos = (catalogo) => { ... }
```

Cuando usarlo

Comprueba que todos los precios son numbers y mayores que 0.

Ejemplo

```
const c=[{precio:10},{precio:0}]
console.log(preciosValidos(c))
```

Resultado en terminal / valor devuelto

false

Ejercicios donde aparece

Parte 3.5.

Cuando se usa junto con otros

every(p => typeof p.precio === "number" && p.precio > 0).

Errores tipicos

Comparar solo p.precio > 0 puede ocultar el requisito del tipo.

Consejo de Miku

Enunciado "todos" -> every; tipo -> typeof.

E. PARTE 4 - PEDIDOS: EL BLOQUE MAS IMPORTANTE

4.1 parsearPedido

Tipo: Parseo string -> objeto

Idea clave: Convierte `cliente|producto:cantidad;producto:cantidad` en `{ cliente, lineas: [{nombre,cantidad}] }`.

Sintaxis

```
export const parsearPedido = (texto) => { ... }
```

Cuando usarlo

Convierte `cliente|producto:cantidad;producto:cantidad` en `{ cliente, lineas: [{nombre,cantidad}] }`.

Ejemplo

```
const pedido = parsearPedido("Ana|Pan:2;Leche:1")
console.log(pedido)
```

Resultado en terminal / valor devuelto

{ cliente: "Ana", lineas: [{ nombre: "Pan", cantidad: 2 }, { nombre: "Leche", cantidad: 1 }] }

Ejercicios donde aparece

Parte 4.1.

Cuando se usa junto con otros

split("|") -> partes[1].split(";") -> cada linea.split(":") -> map -> Number -> objeto final. Usar `const productos = partes[1]` es opcional; `partes[1].split(";")` es equivalente.

Errores tipicos

No olvidar `Number(...)`; split siempre entrega strings. No confundir `;` con el `:`.

Consejo de Miku

Miku: "Piensa en capas: | separa cliente/productos, ; separa productos, : separa nombre/cantidad."

4.2 puedeServirse

Tipo: Comprobacion ALL + busqueda	**Idea clave:** Devuelve true si TODOS los productos del pedido existen y tienen stock suficiente.

Sintaxis

```
export const puedeServirse = (catalogo, pedido) => { ... }
```

Cuando usarlo

Devuelve true si TODOS los productos del pedido existen y tienen stock suficiente.

Ejemplo

```
const c=[{nombre:"Pan",stock:3}]
const p={lineas:[{nombre:"Pan",cantidad:2}]}
console.log(puedeServirse(c,p))
```

Resultado en terminal / valor devuelto

true

Ejercicios donde aparece

Parte 4.2.

Cuando se usa junto con otros

pedido.lineas.every(linea => { const producto = buscarProducto(catalogo,linea.nombre); return producto !== undefined && producto.stock >= linea.cantidad }) No recorres catalogo: recorres `pedido.lineas`. No necesitas comparar con "true" como string.

Errores tipicos

Errores de tu intento anterior: `catalogo.every`, `existeProducto(producto)`, `=` en lugar de `===`, `"true"` string y `stock > 0` insuficiente.

Consejo de Miku

Miku: "Este ejercicio es una plantilla de examen: every + buscarProducto + ===/!== + >=."

4.3 totalPedido

Tipo: Calculo con reduce + find **Idea clave:** Calcula la suma de precio x cantidad de todas las lineas.

Sintaxis

```
export const totalPedido = (catalogo, pedido) => { ... }
```

Cuando usarlo

Calcula la suma de precio x cantidad de todas las lineas.

Ejemplo

```
const c=[{nombre:"Pan",precio:2},{nombre:"Leche",precio:3}]
const p={lineas:[{nombre:"Pan",cantidad:2},{nombre:"Leche",cantidad:1}]}
console.log(totalPedido(c,p))
```

Resultado en terminal / valor devuelto

7

Ejercicios donde aparece

Parte 4.3.

Cuando se usa junto con otros

reduce sobre pedido.lineas + buscarProducto para obtener precio. Si buscas por todo el catalogo dentro de cada linea, el patron sigue siendo correcto: necesitas el objeto para el precio.

Errores tipicos

No olvidar inicial `0`; no sumar stock.

Consejo de Miku

Miku: "Total = reduce. Precio por cantidad = multiplicar dentro de reduce."

4.4 servirPedido

Tipo: map + copia de objeto	**Idea clave:** Devuelve un catalogo NUEVO con el stock descontado solo de los productos pedidos.

Sintaxis

```
export const servirPedido = (catalogo, pedido) => { ... }
```

Cuando usarlo

Devuelve un catalogo NUEVO con el stock descontado solo de los productos pedidos.

Ejemplo

```
const c=[{nombre:"Pan",stock:5,precio:2},{nombre:"Leche",stock:3,precio:3}]
const p={lineas:[{nombre:"Pan",cantidad:2}]}
console.log(servirPedido(c,p))
```

Resultado en terminal / valor devuelto

Pan queda con stock 3; Leche queda con stock 3. El catalogo original sigue intacto.

Ejercicios donde aparece

Parte 4.4.

Cuando se usa junto con otros

catalogo.map(producto => { const linea = pedido.lineas.find(...); if (!linea) return producto; return {...producto, stock: producto.stock - linea.cantidad} }) Combina map + find + spread de objeto.

Errores tipicos

No hagas `producto.stock -= ...` porque mutarias el original; no uses push para construir la salida.

Consejo de Miku

Miku: "map devuelve un catalogo nuevo; spread devuelve un objeto nuevo."

4.5 generarTicket

Tipo: Transformar + unir texto **Idea clave:** Genera un texto con una linea por producto, normalmente con saltos `\n`.

Sintaxis

```
export const generarTicket = (catalogo, pedido) => { ... }
```

Cuando usarlo

Genera un texto con una linea por producto, normalmente con saltos `\n`.

Ejemplo

```
const c=[{nombre:"Pan",precio:2}]
const p={lineas:[{nombre:"Pan",cantidad:2}]}
console.log(generarTicket(c,p))
```

Resultado en terminal / valor devuelto

Una linea de texto por producto; si usas `join("\n")`, cada linea aparece en una linea distinta.

Ejercicios donde aparece

Parte 4.5.

Cuando se usa junto con otros

map(linea=>template literal) + join("\n"). Patron exacto: primero generas cada linea, despues las juntas.

Errores tipicos

No usar `join` sobre el objeto; necesitas un array de strings primero.

Consejo de Miku

Miku: "split rompe texto; join construye texto."

F. PARTE 5 - MUTACION CONTROLADA: COLA, CARRITO E HISTORIAL

5.1 atenderSiguiente

Tipo: Cola FIFO **Idea clave:** Quita y devuelve el primer pedido de la cola.

Sintaxis

```
export const atenderSiguiente = (cola) => cola.shift()
```

Cuando usarlo

Quita y devuelve el primer pedido de la cola.

Ejemplo

```
const cola=["P1","P2"]
console.log(atenderSiguiente(cola))
console.log(cola)
```

Resultado en terminal / valor devuelto

P1 ["P2"]

Ejercicios donde aparece

Parte 5.1 y 6.1.

Cuando se usa junto con otros

shift + length/estado de la cola.

Errores tipicos

Usar pop rompe FIFO.

Consejo de Miku

Miku: "Primero en entrar, primero en salir."

5.2 agregarUrgente

Tipo: Insertar al principio **Idea clave:** Coloca un pedido al principio para darle prioridad.

Sintaxis

```
export const agregarUrgente = (cola, pedido) => cola.unshift(pedido)
```

Cuando usarlo

Coloca un pedido al principio para darle prioridad.

Ejemplo

```
const cola=["Normal1","Normal2"]
agregarUrgente(cola,"URGENTE")
console.log(cola)
```

Resultado en terminal / valor devuelto

["URGENTE", "Normal1", "Normal2"]

Ejercicios donde aparece

Parte 5.2.

Cuando se usa junto con otros

unshift.

Errores tipicos

Usar push no lo hace urgente.

Consejo de Miku

Miku: "Urgente entra por delante."

5.3 agregarAlCarrito

Tipo: Mutacion + historial **Idea clave:** Añade un producto al final y registra la accion para poder deshacerla.

Sintaxis

```
export const agregarAlCarrito = (carrito, historial, producto) => { ... }
```

Cuando usarlo

Añade un producto al final y registra la accion para poder deshacerla.

Ejemplo

```
const c=["Pan"]; const h=[]
agregarAlCarrito(c,h,"Leche")
console.log(c,h)
```

Resultado en terminal / valor devuelto

["Pan","Leche"] y el historial contiene la accion de añadir

Ejercicios donde aparece

Parte 5.3.

Cuando se usa junto con otros

push en carrito + push en historial. Registrar la accion exacta antes de perder la informacion necesaria para deshacer.

Errores tipicos

No olvidar actualizar ambos arrays.

Consejo de Miku

Miku: "Carrito y memoria tienen que quedar sincronizados."

5.4 quitarDelCarrito

Tipo: Busqueda + splice + historial **Idea clave:** Quita la primera aparicion del producto y registra la accion.

Sintaxis

```
export const quitarDelCarrito = (carrito, historial, producto) => { ... }
```

Cuando usarlo

Quita la primera aparicion del producto y registra la accion.

Ejemplo

```
const c=["Pan","Leche","Pan"]; const h=[]
quitarDelCarrito(c,h,"Pan")
console.log(c)
```

Resultado en terminal / valor devuelto

["Leche","Pan"]

Ejercicios donde aparece

Parte 5.4.

Cuando se usa junto con otros

indexOf -> si no es -1 -> splice(indice,1) -> registrar accion. No uses filter si el requisito es quitar solo la primera aparicion y mutar el array.

Errores tipicos

Comprobar -1 antes de splice.

Consejo de Miku

Miku: "Borrar uno concreto = indexOf + splice."

5.5 deshacer

Tipo: Pila LIFO **Idea clave:** Revierte la ultima accion registrada. `historial.pop()` hace LIFO.

Sintaxis

```
export const deshacer = (carrito, historial) => { ... }
```

Cuando usarlo

Revierte la ultima accion registrada. `historial.pop()` hace LIFO.

Ejemplo

```
const c=["Pan"]; const h=[{tipo:"añadir",producto:"Leche"}]
deshacer(c,h)
console.log(c,h)
```

Resultado en terminal / valor devuelto

Carrito vuelve a ["Pan"] y el historial queda vacio.

Ejercicios donde aparece

Parte 5.5.

Cuando se usa junto con otros

pop del historial -> segun tipo, deshacer con indexOf/lastIndexOf/splice o push. Guardar suficiente informacion en cada entrada del historial.

Errores tipicos

Confundir FIFO con LIFO; `pop` es la clave.

Consejo de Miku

Miku: "Deshacer siempre mira primero la ultima accion."

G. PARTE 6 - INTEGRACION

6.1 procesarCola

Tipo: Integracion + while

Idea clave: Procesa pedidos hasta vaciar la cola. Cada vuelta atiende el primero, comprueba si se puede servir y, si procede, actualiza el catalogo.

Sintaxis

```
export const procesarCola = (catalogo, cola) => { while (...) { ... } return ... }
```

Cuando usarlo

Procesa pedidos hasta vaciar la cola. Cada vuelta atiende el primero, comprueba si se puede servir y, si procede, actualiza el catalogo.

Ejemplo

```
while (cola.length > 0) { const pedido = atenderSiguiente(cola); if (puedeServirse(catalogo,pedido)) catalogo = servirPedido(catalog
```

Resultado en terminal / valor devuelto

La cola termina vacia; el catalogo refleja las ventas servibles.

Ejercicios donde aparece

Parte 6.1.

Cuando se usa junto con otros

while + shift + every/find + map. El while debe tener una condicion que avance: la cola disminuye gracias a shift.

Errores tipicos

No olvidar reasignar `catalogo = servirPedido(...)` porque servirPedido devuelve un catalogo nuevo.

Consejo de Miku

Miku: "Parte 6.1 no inventa nada: conecta 5.1 + 4.2 + 4.4."

6.2 productosVendidos

Tipo: Transformacion encadenada **Idea clave:** Devuelve nombres unicos de productos vendidos, ordenados.

Sintaxis

```
export const productosVendidos = (pedidos) => { ... }
```

Cuando usarlo

Devuelve nombres unicos de productos vendidos, ordenados.

Ejemplo

```
const pedidos=[{lineas:[{nombre:"Pan"},{nombre:"Leche"}]},{lineas:[{nombre:"Pan"}]}]
console.log(productosVendidos(pedidos))
```

Resultado en terminal / valor devuelto

["Leche", "Pan"]

Ejercicios donde aparece

Parte 6.2.

Cuando se usa junto con otros

map -> flat -> map -> filter con indexOf -> sort localeCompare. Tambien puede resolverse con otros enfoques, pero en el ejercicio debes reconocer la cadena pedida.

Errores tipicos

No olvidar eliminar duplicados antes de ordenar o, si lo haces despues, mantener el resultado correcto.

Consejo de Miku

Miku: "Mira la frase del enunciado y divide el trabajo en mini transformaciones."

6.3 graficoStock

Tipo: Construccion de strings **Idea clave:** Genera lineas de texto con bloques repetidos segun stock.

Sintaxis

```
export const graficoStock = (catalogo) => { ... }
```

Cuando usarlo

Genera lineas de texto con bloques repetidos segun stock.

Ejemplo

```
const stock=5
const barra = new Array(stock).fill("■").join("")
console.log(barra)
```

Resultado en terminal / valor devuelto

■■■■■

Ejercicios donde aparece

Parte 6.3.

Cuando se usa junto con otros

new Array(n) + fill + join; Math.floor si el ejercicio define una escala. Si n es 0, `new Array(0).fill(...).join("")` produce string vacio.

Errores tipicos

No uses repeat si el ejercicio esta diseñado para practicar fill/join.

Consejo de Miku

Miku: "Piensa barra = array de bloques -> fill -> join."

H. COMBINACIONES QUE DEBES RECONOCER

filter -> map

Idea: Selecciono y despues transformo

```
catalogo.filter(p => p.stock > 0).map(p => p.nombre)
```

Resultado: ["Zelda", "Mario"]

Ejercicios: Ejercicios 2.1 y 2.2: agotados, canciones largas, aprobados.

Consejo: Primero decide quien se queda; luego que dato quieres sacar de ellos.

map -> sort

Idea: Extraigo una lista y la ordeno

```
catalogo.map(p => p.nombre).sort((a,b) => a.localeCompare(b))
```

Resultado: ["Ana", "Mario", "Zelda"]

Ejercicios: 2.2 - nombresOrdenados.

Consejo: Si la salida debe ser solo strings, esta cadena es natural.

filter -> map -> join

Idea: Creo texto a partir de elementos que cumplen

```
productos.filter(p => p.stock > 0).map(p => p.nombre).join(", ")
```

Resultado: Zelda, Mario

Ejercicios: Tickets, listados, mensajes.

Consejo: Muy util cuando la salida final es un string.

map -> flat -> map

Idea: Transformo arrays anidados y los aplano

```
pedidos.map(p => p.lineas).flat().map(l => l.nombre)
```

Resultado: ["Pan","Leche","Pan"]

Ejercicios: 2.2 - productosVendidos.

Consejo: Divide una tarea dificil en etapas pequeñas.

indexOf -> splice

Idea: Localizo y borro

```
const i = arr.indexOf(x); if (i !== -1) arr.splice(i,1)
```

Resultado: Array con una aparicion menos

Ejercicios: 2.2 - quitarDelCarrito/deshacer.

Consejo: Es una de las parejas mas importantes de la Parte 5.

find -> propiedad

Idea: Busco objeto y uso un dato

```
const p = catalogo.find(x => x.nombre === "Pan"); console.log(p.precio)
```

Resultado: 2

Ejercicios: 2.1 biblioteca; 2.2 buscarProducto, totalPedido.

Consejo: Comprueba `p !== undefined` si existe posibilidad de no encontrarlo.

every -> find

Idea: Todos los elementos pedidos deben poder resolverse

```
pedido.lineas.every(l => { const p = buscarProducto(catalogo,l.nombre); return p !== undefined && p.stock >= l.cantidad })
```

Resultado: true/false

Ejercicios: 2.2 - puedeServirse.

Consejo: Esta es la combinacion clave de 4.2.

reduce -> find

Idea: Acumulo usando datos que estan en otro array

```
pedido.lineas.reduce((total,l) => { const p=buscarProducto(catalogo,l.nombre); return total+p.precio*l.cantidad },0)
```

Resultado: Total numerico

Ejercicios: 2.2 - totalPedido.

Consejo: Es una combinacion de alto valor en examen.

copy -> sort

Idea: Ordeno sin mutar

```
const ordenados = [...catalogo].sort((a,b) => a.precio - b.precio)
```

Resultado: Nuevo array ordenado; original intacto

Ejercicios: 2.2 - ordenarPorPrecio.

Consejo: Si te piden "sin modificar el original", busca una copia antes de sort.

map -> join("\n")

Idea: Construyo un bloque de texto por lineas

```
lineas.map(l => `...`).join("\n")
```

Resultado: String con saltos de linea

Ejercicios: 2.2 - generarTicket.

Consejo: Piensa "una cadena por elemento y despues las coso".

I. CHECKLIST FINAL DE EXAMEN

Antes de entregar una funcion, comprueba:

1. Los parametros tienen exactamente el nombre y orden que pide el profesor.
2. La funcion esta exportada si el ejercicio lo necesita.
3. He usado `const`/`let` y funciones flecha segun las normas del ejercicio.
4. He elegido el array correcto para recorrer. En 4.2: `pedido.lineas`, no `catalogo`.
5. He elegido el metodo por el significado del enunciado: todos/some/map/filter/find/reduce...
6. Si ordeno, he usado comparador numerico o `localeCompare` y he copiado antes si no debo mutar.
7. Si borro un elemento concreto: `indexOf/findIndex` + `splice`.
8. En pedidos: `split` para leer el texto, `join` para construir texto.
9. En 4.4: spread de objeto para no modificar el objeto original.
10. En 6.1: el while realmente vacia la cola.
11. He revisado que `find` puede devolver undefined y `findIndex` puede devolver -1.
12. He probado `node pruebas.js` despues de cada bloque antes de seguir.

Mini chuleta de terminal

```
node pruebas.js
node main.js
```

Resultados que debes saber interpretar

```
true / false      -> booleano
undefined         -> no se encontro / no se devolvio valor
-1                -> findIndex/indexOf no encontraron
[]                -> array vacio / no hay elementos
{ ... }           -> objeto
numero            -> total, posicion o cantidad
string            -> ticket / mensaje / grafico
```

Regla final de Miku

Lee el verbo del enunciado antes de pensar en la sintaxis. La mayoria de los ejercicios 2.2 se resuelven reconociendo una combinacion de patrones que ya conoces.

Nota de alcance

Este anexo mantiene como prioridad la terminologia y los patrones de tus materiales. El nuevo UT 2.2 incluye, entre otros, las familias agregar/quitar, recorrer, buscar, transformar y extras; destaca la mutacion de push/pop/shift/unshift/splice/sort/reverse/fill/copyWithin y las operaciones nuevas como findLastIndex, reduceRight, flatMap y toSorted. El ejercicio 2.2 sigue siendo la referencia practica para el examen.